DL Architectures & Generative Techniques

Master Summary

Stefan Eder

Based on *Deep Learning and Neural Networks* SS25

How to use this document. This is a structured summary built around *running text*. Explanations, derivations, and logical flow between ideas live in the prose — colored boxes are reserved for the few things that must stand out at a glance.

- **Running text** — read for understanding, derivations, and context.
- **Colored boxes** — scan for key definitions, theorems, examples, and exam notes.

Boxes lose their impact when overused. When in doubt, write prose.

Compiled June 24, 2026

Contents

1

Image Classification

Computer vision tasks span a spectrum of difficulty. In *image classification* a single class label is predicted for the most prominent object in an image. *Object detection* additionally localizes each instance with a bounding box, *semantic segmentation* assigns class labels per pixel, and *instance segmentation* combines both. This chapter traces the rapid evolution of CNN-based image classification, from AlexNet's breakthrough in 2012 to transformer-based approaches a decade later. The top-5 error on ImageNet fell from roughly 26% to under 3% in five years, eventually reaching and surpassing the human error rate of $\approx 3.5\%$.

1.1 The ImageNet Benchmark

The ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) became the standard yardstick for image classification progress. Each image is labeled with its dominant object class; the dataset covers 1000 classes. Two evaluation protocols are used: **top-1 error** — the highest-ranked prediction must be correct — and **top-5 error** — the correct class must appear within the five highest-ranked predictions. Most papers report top-5 error, which is less sensitive to label ambiguity.

Before 2012 the leading methods relied on hand-crafted features (SIFT, HoG, Haar-like features); CNNs changed that completely. A key additional observation is that features learned for ImageNet transfer remarkably well to other datasets: pre-training on ImageNet and fine-tuning on a smaller target dataset consistently outperforms training from scratch, especially when the target set is small.

Top-k Error

A prediction is correct under the *top-k criterion* if the true class label appears among the k highest-scoring predictions of the model. Top-5 error is the fraction of test images for which this criterion fails.

1.2 AlexNet (2012)

AlexNet (Krizhevsky et al., 2012) won ILSVRC-2012 with a top-5 error of **15.3%**, far below the runner-up's 26.2%, and marked the start of the deep-learning era in vision.

The network consists of 5 convolutional layers followed by 3 fully-connected layers, totaling 60 million parameters and 650k neurons. The input is a $224 \times 224 \times 3$ image. Training ran for 5–6 days on two GTX 580 3 GB GPUs; GPU memory was the binding constraint.

AlexNet Architecture

- **8 learned layers**: 5 convolutional + 3 fully-connected
- **Parameters**: $\approx 60 \times 10^6$
- **Input**: $224 \times 224 \times 3$
- **Output**: 1000-way softmax

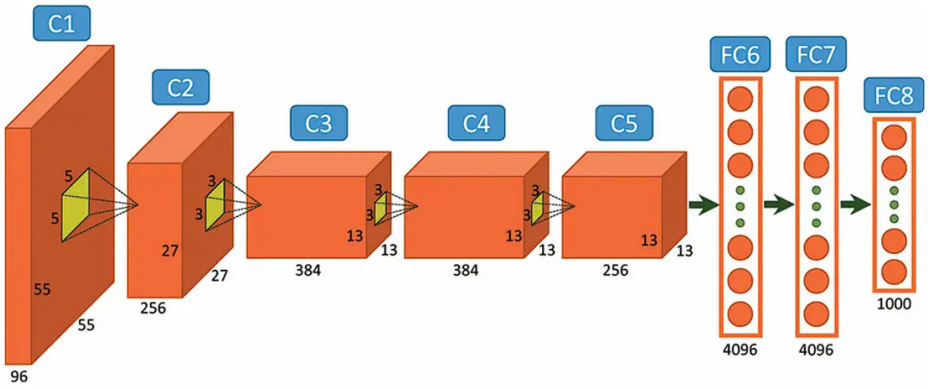

Figure 1.1: AlexNet: five convolutional layers (with ReLU, overlapping max-pooling and LRN) feeding three fully-connected layers into a 1000-way softmax. The original split across two GPUs is what gives the diagram its characteristic two-stream layout.

Four innovations drove AlexNet's success. **ReLU activations** replaced tanh, providing substantial speed-up in training. **Dropout** in the fully-connected layers acted as a strong regularizer against overfitting. **Overlapping max-pooling** used 3×3 windows with stride 2. Finally, **Local Response Normalization** (LRN) provided competitive lateral inhibition across neighboring feature maps, loosely inspired by cortical inhibition — though later work showed LRN does not actually help and it was abandoned.

LRN is obsolete

Local Response Normalization was used in AlexNet and ZfNet but shown by VGGNet not to improve performance. Batch Normalization replaced it entirely.

1.3 ZfNet (2013)

ZfNet (Zeiler & Fergus, 2013) achieved **13.5%** top-5 error. Its main contribution was a *deconvolution-based visualization* technique: by reversing the forward pass through convolution, pooling, and ReLU layers, one can project feature maps back to pixel space and inspect what each filter responds to.

This analysis revealed that AlexNet's first-layer 11×11 kernels with stride 4 created aliasing artifacts and many "dead" (never-activated) filters. ZfNet replaced them with 7×7 kernels and stride 2, yielding cleaner low-level features and better accuracy with comparable architecture depth. ZfNet also performed the first systematic *transfer learning* experiments, showing that fine-tuning a pre-trained model on CalTech-101/256 quickly surpassed previous state-of-the-art on those benchmarks.

1.4 VGGNet (2014)

VGGNet (Simonyan & Zisserman, 2014) won the localization task and placed second in classification at ILSVRC-2014 with **7.3%** top-5 error. It was directly motivated by ZfNet's finding that large kernels are suboptimal.

Receptive-Field Equivalence of Stacked 3×3 Kernels

Two stacked 3×3 convolutional layers (stride 1) cover the same receptive field as a single 5×5 layer, and three 3×3 layers equal one 7×7 layer. The stacked design is strictly better: it introduces more non-linearities and has *fewer* parameters ($2 \times 3^2C^2 = 18C^2$ vs. $5^2C^2 = 25C^2$ for C channels).

VGGNet uses *exclusively* 3×3 convolutions with stride 1. The two most common variants are **VGG-16** (best single-crop top-5) and **VGG-19** (marginally better with multi-crop evaluation). The study also confirmed that LRN does not help and that 1×1 convolutions alone (without depth increase) do not improve performance.

A serious flaw: the first fully-connected layer takes the $7 \times 7 \times 512$ feature map and maps it to 4096 units *without* spatial weight sharing, creating a single layer with $> 100 \times 10^6$ parameters — more than 80% of the entire network. This makes VGGNet prone to overfitting and expensive to deploy.

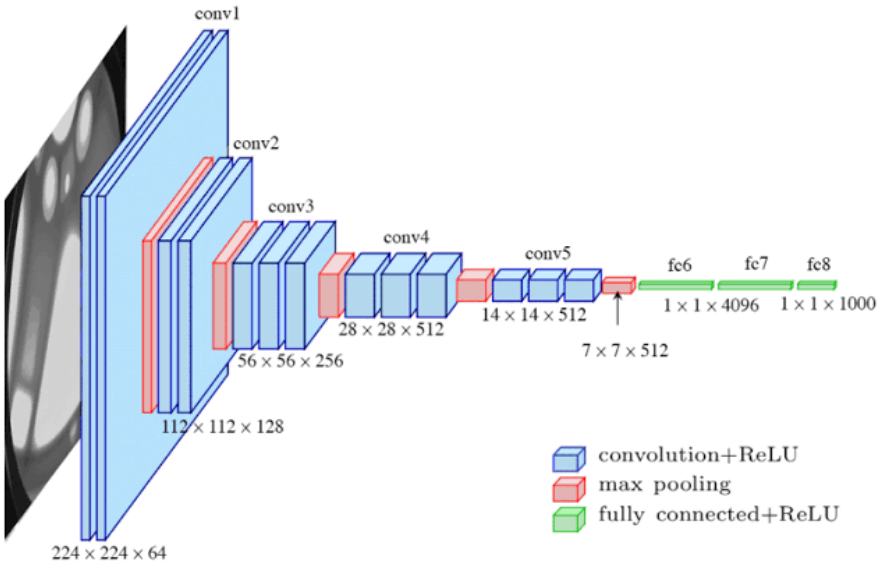

Figure 1.2: VGG-16: a uniform stack of 3×3 convolution blocks (each followed by 2×2 max-pooling) that progressively halves the spatial resolution while doubling the channel count, ending in three large fully-connected layers. The width labels ($224 \times 224 \times 64$ down to $7 \times 7 \times 512$) make the down-sampling schedule explicit.

1.5 Network in Network (2014)

Network in Network (NiN; Lin et al., 2014) did not enter ILSVRC but introduced two ideas that shaped all subsequent architectures.

MLPconv layers. Replace the linear filter in a standard convolution with a small MLP applied to each local patch. In practice this equals a $k \times k$ convolution followed by two 1×1 convolutions with ReLU — increasing non-linear representational power without enlarging the receptive field.

Global Average Pooling (GAP). Instead of flattening spatial feature maps into a vector and feeding fully-connected layers, the final convolutional layer produces one feature map per class; GAP then reduces each map to a scalar by averaging.

Global Average Pooling

Given a feature tensor of shape $H \times W \times C$, GAP outputs a C-dimensional vector where the c-th entry is $\frac{1}{HW} \sum_{i,j} x_{i,j,c}$. Applied to a C-class classification head this yields *zero* extra parameters while providing spatial translation invariance and making each channel directly interpretable as a class confidence map.

Intuition & Mental Model

GAP can be seen as enforcing the network to make its last feature maps "honest": channel c must actually encode evidence for class c, because the spatial average of that channel is the final logit.

1.6 GoogLeNet / Inception v1 (2014)

GoogLeNet (Szegedy et al., 2014) won ILSVRC-2014 classification with **6.7%** top-5 error using 22 layers and only **6.9 million parameters** — less than 6% of VGGNet's 122 million. The key insight: 85% of AlexNet/VGG parameters lived in the fully-connected layers and mainly contributed to overfitting; replacing them with GAP plus one FC layer saves $> 99\%$ of those parameters.

1.6.1 Inception Modules

The building block is the *Inception module*, which processes the input in parallel through 1×1, 3×3, and 5×5 convolutions as well as a max-pooling branch, then concatenates all outputs along the channel dimension. This lets the network choose its own filter sizes rather than committing to one scale.

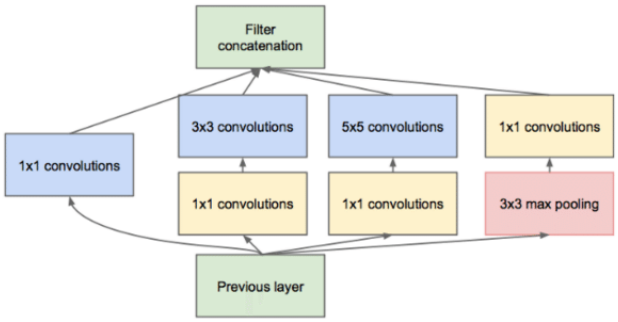

Figure 1.3: The Inception module: parallel 1×1, 3×3, 5×5 convolution branches and a pooling branch, with 1×1 convolutions used as cheap dimensionality reductions before the larger kernels. The branch outputs are concatenated along the channel dimension.

The naive version has two problems: (i) the channel dimension grows at every module since pooling cannot reduce channels, and (ii) 5×5 convolutions on a large feature map are expensive. Both are solved by **bottleneck 1×1 convolutions**: placed before the 3×3 and 5×5 branches they reduce the input channel count, and placed after the pooling branch they prevent channel explosion.

Bottleneck Parameter Savings (Inception 4a example)

Without bottleneck:

3×3 branch : $3^2 \times 480 \times 208 \approx 898$k params; 5×5 branch : $5^2 \times 480 \times 48 \approx 576$k params.

With 1×1 bottleneck (96 and 16 intermediate channels respectively):

$$3\times3 : 480 \times 96 + 3^2 \times 96 \times 208 \approx 225\text{k}; \quad 5\times5 : 480 \times 16 + 5^2 \times 16 \times 48 \approx 27\text{k}.$$

Savings of roughly $4\times$ and $20\times$ per branch.

Because the network is 22 layers deep, the gradient signal can weaken before reaching the early layers (vanishing gradient). GoogLeNet addresses this with two **auxiliary loss heads** attached to intermediate layers during training; they are discarded at inference.

1.7 Batch-Normalized Inception / Inception v2 (2015)

The 2015 Inception variant integrated Batch Normalization (BN) after every convolution, achieving **4.8%** top-5 error. The headline result is speed: BN reduced the number of training steps needed to reach 72% accuracy by a factor of **14**. BN also replaced Dropout as the primary regularizer in this architecture. An improved down-sampling block ran max-pooling and a stride-2 bottleneck convolution in parallel, then concatenated the results.

1.8 Highway Networks (2015)

Highway Networks (Srivastava et al., 2015) asked: can gradient descent train networks with *hundreds* of layers? The answer is yes, with *gating* borrowed from LSTMs.

A standard feedforward layer computes $\mathbf{h} = H(\mathbf{x}; \mathbf{W}_H)$. A Highway layer adds learnable transform and carry gates:

Highway Network Layer

$$\mathbf{h} = H(\mathbf{x}; \mathbf{W}_H) \odot T(\mathbf{x}; \mathbf{W}_T) + \mathbf{x} \odot (1 - T(\mathbf{x}; \mathbf{W}_T)),$$

where $T(\mathbf{x}; \mathbf{W}_T) = \sigma(\mathbf{W}_T\mathbf{x}+\mathbf{b}_T)$ is the *transform gate* and $\odot$ is element-wise multiplication. Initializing $\mathbf{b}_T$ to a large negative value biases the network to carry information forward early in training, allowing gradients to flow unimpeded.

100-layer Highway Networks were successfully trained on CIFAR-10 and CIFAR-100, directly motivating residual networks.

1.9 ResNet (2015)

ResNet (He et al., 2015) won every track of ILSVRC-2015, achieving **3.57%** top-5 error on classification — below the human error rate — and won 194 of 200 categories in detection. It became the dominant backbone for downstream vision tasks.

1.9.1 Residual Learning

The central observation: plain networks degrade in accuracy as depth increases beyond a certain point, even on the training set, suggesting an optimization problem rather than overfitting. If a shallower network achieves accuracy A, a deeper one should be able to match it by learning identity mappings in the added layers — yet this does not happen.

The fix: instead of asking k stacked layers to approximate the target function $F(\mathbf{x})$ directly, ask them to approximate the *residual* $F(\mathbf{x}) - \mathbf{x}$, then add the input back via a *skip connection*.

Residual Block

$$\mathbf{h} = \mathcal{F}(\mathbf{x}; \{\mathbf{W}_i\}) + \mathbf{x},$$

where $\mathcal{F}$ is the residual mapping (two or three convolutional layers) and $\mathbf{x}$ is the identity shortcut. *No extra parameters* are added by the shortcut.

Intuition & Mental Model

If the optimal function is close to identity, it is easier to push $\mathcal{F}$ toward zero than to learn an identity from scratch through a stack of non-linear layers. Gradients can also flow directly through the shortcut to lower layers, mitigating vanishing gradients.

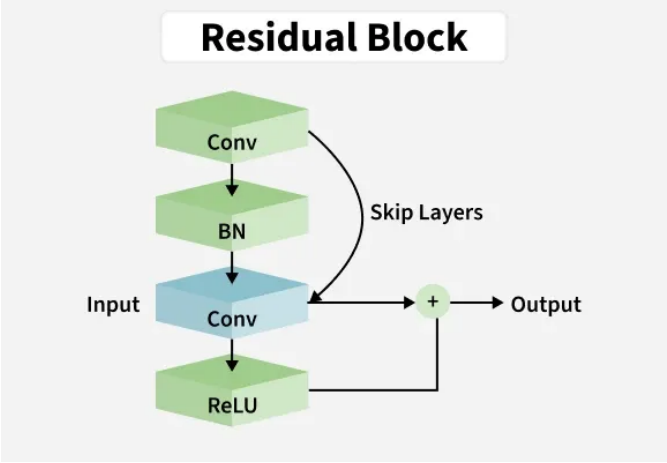

Figure 1.4: A residual block: the input $\mathbf{x}$ is added back to the output of a small stack of weight layers via an identity skip connection, so the block learns the residual $\mathcal{F}(\mathbf{x})$ rather than the full mapping $\mathcal{F}(\mathbf{x}) + \mathbf{x}$.

1.9.2 Bottleneck Blocks

For ResNet-50 and deeper, the basic $3 \times 3 / 3 \times 3$ block is replaced by a *bottleneck* block: 1×1 (reduce channels) $\to$ 3×3 $\to$ 1×1 (restore channels). For example, ResNet-50 processes $256 \to 64 \to 64 \to 256$ channels per block. This keeps computation comparable to ResNet-34 while tripling depth.

1.9.3 Projection Shortcuts

When a skip connection crosses a change in spatial resolution or channel count (e.g., between stages), the identity shortcut is replaced by a 1×1 convolution with stride 2, matching dimensions for the addition.

1.9.4 Variants and FLOPs

Variant	Parameters	FLOPs
ResNet-18	11.7×10^6	1.8×10^9
ResNet-34	21.8×10^6	3.6×10^9
ResNet-50	25.6×10^6	3.8×10^9
ResNet-101	44.5×10^6	7.6×10^9
ResNet-152	60.2×10^6	11.3×10^9

ResNets are more popular than Inception as backbones for transfer learning because their generic, modular design is easier to adapt and modify.

Batch Normalization is Critical

Batch Normalization is applied after every convolution and before the activation. Without BN, training a 50+-layer ResNet is infeasible. Recent work (Huang et al., 2019) showed that self-normalizing activations can substitute for BN and skip connections simultaneously, but BN + skip connections remains the standard recipe.

1.10 Pre-activation ResNet / ResNet-v2 (2016)

He et al. studied identity mapping variants and proposed moving BN and ReLU *before* the weight layers (pre-activation) rather than after. The identity shortcut then passes completely unmodified:

$$\mathbf{h} = \mathbf{x} + \mathcal{F}(\mathrm{BN}(\mathrm{ReLU}(\mathbf{x}))) \,.$$

On CIFAR-10, ResNet-1001 improves from 7.61% (original) to **4.92%** (pre-activation) with no parameter change, demonstrating that the exact placement of normalization matters for very deep networks.

1.11 ResNeXt (2016)

ResNeXt (Xie et al., 2016) achieved **3.0%** top-5 error at ILSVRC-2016 (second place; first was an ensemble at 2.9%). It introduces *grouped convolutions* controlled by a *cardinality* parameter C: instead of one large 3×3 convolution operating on all channels, C independent groups each operate on $1/C$ of the channels.

Grouped Convolution Parameter Count

For a 3×3 convolution with C_{in} input and C_{out} output channels split into C groups:

$$\text{Parameters} = C \times \frac{C_{\text{in}}}{C} \times 3^2 \times \frac{C_{\text{out}}}{C} = \frac{3^2 \, C_{\text{in}} \, C_{\text{out}}}{C}.$$

Cardinality $C = 32$ reduces the 3×3 cost by $32\times$ while doubling the 1×1 costs, keeping the total roughly constant.

Grouped convolutions date back to AlexNet, where they were a GPU memory workaround. ResNeXt repurposed them as an intentional architectural design principle. The sparseness ratio of each block is $1 - 1/C$.

1.12 DenseNet (2016)

DenseNet (Huang et al., 2016) extends the skip-connection idea to its logical extreme: *every layer receives the concatenated feature maps of all preceding layers.* Layer l takes $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}$ as input and produces an output of *growth rate* G channels.

Dense Block Connectivity

$$\mathbf{x}_l = H_l([\mathbf{x}_0,\, \mathbf{x}_1,\, \ldots,\, \mathbf{x}_{l-1}])\,,$$

where $[\cdot]$ denotes channel-wise concatenation and $H_l : \mathbb{R}^{G(l-1)} \to \mathbb{R}^G$ is a BN→ReLU→Conv block.

Dense connectivity enables feature reuse, provides implicit deep supervision (gradients reach early layers directly through concatenation paths), and reduces overfitting. DenseNets achieve better top-1 accuracy than ResNets of comparable parameter count: DenseNet-169 beats ResNet-50 on ImageNet while using fewer parameters.

1.13 Squeeze-and-Excitation Network (SENet, 2017)

SENet (Hu et al., 2017) won ILSVRC-2017 — the final year — with **2.2%** top-5 error. It adds a lightweight *channel-attention* module to any existing block.

SE Block

Given a feature map $\mathbf{U} \in \mathbb{R}^{H \times W \times C}$:

1. **Squeeze**: Global average pooling → descriptor $\mathbf{z} \in \mathbb{R}^C$.
2. **Excitation**: $\mathbf{s} = \sigma(\mathbf{W}_2\ \mathrm{ReLU}(\mathbf{W}_1\mathbf{z}))$, with $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$ (reduction ratio r, typically 16).
3. **Scale**: $\tilde{\mathbf{u}}_c = s_c \cdot \mathbf{u}_c$ (channel-wise scaling).

The SE block adds only $\frac{2C^2}{r}$ parameters but lets the network re-weight channels based on global image statistics. It can be attached to Inception modules (SE-Inception) or ResNet blocks (SE-ResNet) with minimal overhead.

1.14 MobileNet V2 (2018)

MobileNet V2 targets *mobile inference*: competitive accuracy with minimal multiply-accumulate operations. The core building block is the *inverted residual with linear bottleneck*.

1.14.1 Depthwise Separable Convolutions

A standard $R \times R$ convolution with C_l input and C_{l+1} output channels costs $R^2 C_l C_{l+1}$ parameters. *Depthwise separable convolutions* factor this into:

1. A *depthwise* $R \times R$ convolution applied independently per channel: $R^2 C_l$ params.
2. A *pointwise* 1×1 convolution mixing channels: $C_l C_{l+1}$ params.

Total: $R^2 C_l + C_l C_{l+1}$, vs. $R^2 C_l C_{l+1}$ — roughly $R^2\times$ cheaper. Depthwise separable convolutions are the extreme case of grouped convolutions (cardinality $= C_l$).

1.14.2 Inverted Residuals

Standard ResNet bottlenecks shrink channels, apply a 3×3, then expand. MobileNet V2 *inverts* this: expand (cheap 1×1) $\rightarrow$ depthwise 3×3 $\rightarrow$ project back to small dimension (1×1). The skip connection runs between the low-dimensional *bottleneck* representations, not across the high-dimensional expansion.

No ReLU after Bottleneck Projection

The final 1×1 projection uses a *linear* activation. Applying ReLU to a low-dimensional representation collapses the information irreversibly (it sets half the values to zero while the subspace has little redundancy).

1.15 EfficientNet (2019)

EfficientNet (Tan & Le, 2019) asks: given a fixed compute budget, how should one scale a CNN along depth d, width w, and input resolution r? The three dimensions are not independent: a higher-resolution image needs a deeper network (larger receptive field) and a wider one (more fine-grained features).

Compound Scaling

Let ϕ be a user-specified resource coefficient. Scale:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi,$$

subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, with $\alpha, \beta, \gamma \geq 1$. Since FLOPs $\propto d \cdot w^2 \cdot r^2$, total FLOPs scale as 2^ϕ. Constants α, β, γ are found once by a small grid search on the baseline architecture (EfficientNet-B0); ϕ is then varied to obtain B1–B7.

The baseline B0 uses inverted residuals (MobileNet V2) with SE blocks (SENet). **EfficientNet-B7** achieves **84.4%** top-1 accuracy on ImageNet with 66M parameters — 8.4× smaller and 6.1× faster than the previous state-of-the-art GPipe. B0 already beats ResNet-50 with 4.9× fewer parameters and 11× fewer FLOPs.

1.16 Vision Transformer (ViT, 2020)

ViT (Dosovitskiy et al., 2020) applies a pure Transformer — no convolutions — to image classification by treating an image as a sequence of patches.

An image is divided into non-overlapping 16×16-pixel patches, each linearly projected to a d-dimensional embedding; a learnable `[CLS]` token is prepended and positional encodings are added. The sequence is processed by a standard Transformer encoder.

Self-Attention (single head)

Given input set $\mathbf{H}^{(0)} \in \mathbb{R}^{d \times T}$ (T patches):

$$\mathbf{Q}^{(l)} = \mathbf{W}_Q \mathbf{H}^{(l-1)}, \quad \mathbf{K}^{(l)} = \mathbf{W}_K \mathbf{H}^{(l-1)}, \quad \mathbf{V}^{(l)} = \mathbf{W}_V \mathbf{H}^{(l-1)},$$

$$\mathbf{H}^{(l)} = \text{softmax}\left(\frac{\mathbf{Q}^{(l)} {\mathbf{K}^{(l)}}^\mathsf{T}}{\sqrt{d_k}}\right) \mathbf{V}^{(l)}.$$

The attention matrix is $T \times T$, so complexity is $\mathcal{O}(T^2)$ — quadratic in the number of patches (and prohibitive for full-resolution images).

ViT requires large-scale pre-training

Unlike CNNs, ViT lacks the spatial inductive biases (locality, translation equivariance) that make CNNs data-efficient. ViT only matches or beats CNNs when pre-trained on very large datasets (JFT-300M, 300 million images) and then fine-tuned on ImageNet.

1.17 Self-Supervised Learning (from ∼2020)

Self-supervised pre-training learns representations from *unlabeled* data using a *pretext task* that requires semantic understanding. Two main paradigms have emerged.

1.17.1 Contrastive Learning (e.g., SimCLR)

Two different augmentations of the same image form a *positive pair*; augmentations from different images form *negative pairs*. The network is trained to maximize similarity of positive pairs and minimize similarity of negatives.

InfoNCE Loss (NT-Xent)

$$\mathcal{L}\Big((\mathbf{h}_0, \mathbf{h}_j),\, \{\mathbf{h}_l\}_{l=1}^{L}\Big) = -\log \frac{\exp\big(\cos(\mathbf{h}_0, \mathbf{h}_j)/\tau\big)}{\sum_{l=1}^{L} \exp\big(\cos(\mathbf{h}_0, \mathbf{h}_l)/\tau\big)},$$

where $\cos(\mathbf{h}_i, \mathbf{h}_j) = \frac{\mathbf{h}_i^\mathsf{T} \mathbf{h}_j}{\|\mathbf{h}_i\| \|\mathbf{h}_j\|}$ is cosine similarity and $\tau > 0$ is a temperature hyperparameter.

1.17.2 Masked Image Modeling (e.g., MAE)

Random patches of an image are masked; the model must reconstruct them. Analogous to BERT's masked language modeling, this forces the network to learn global semantic structure.

1.18 Key Architectural Lessons

The drop from ∼26% to ∼2% top-5 error between 2012 and 2017 was driven by a small set of recurring design principles:

1. **Small kernels**: Stack 3×3 convolutions instead of large ones — same receptive field, more non-linearities, fewer parameters.
2. **Skip connections**: Highway gates or identity shortcuts enable gradient flow and training of very deep networks.
3. **Global Average Pooling**: Replace FC layers with GAP to eliminate spatial-dimension parameters and reduce overfitting.
4. 1×1 **convolutions**: Change channel count without altering spatial dimensions; enable bottlenecks.
5. **Bottleneck blocks**: Compress channels before costly 3×3 operations, simulating sparse connectivity on dense hardware.
6. **Grouped / depthwise convolutions**: Redistribute computation toward channel mixing; enable extreme parameter efficiency (MobileNet).
7. **Compound scaling**: Depth, width, and resolution are interdependent; balance them together for optimal efficiency (EfficientNet).
8. **Attention**: Channel-wise (SE) and spatial (Transformer) attention lets the network focus resources on the most informative parts of the input.

2

Semantic Segmentation

The four fundamental vision tasks form a natural ladder of difficulty. *Image classification* hands back a single label for the whole picture. *Object detection* draws a box around each thing it finds. *Semantic segmentation* goes all the way down to the pixel: every single pixel gets a class label (road, car, pedestrian, sky, ...). *Instance segmentation* does the same but also keeps the individual objects apart — this cow versus that cow — and we get to it later via Mask R-CNN.

The defining feature of semantic segmentation is that the output has the *same spatial size as the input*. We are not collapsing the image down to a single decision; we are producing a full-resolution label map. That one requirement — keep (or recover) the resolution — is what makes the whole chapter interesting, because everything a classification CNN does (pooling, striding) is busy throwing resolution away. The art is in getting it back.

2.1 What we are optimizing, and how we measure it

The natural thing to ask of a segmentation is: *how many pixels did we get right?* Raw pixel accuracy is a weak metric, though, because most pixels in a typical scene are easy background. The standard score is instead the **Intersection over Union** (IoU), also called the *Jaccard index*. For two pixel sets $\mathcal{A}$ (predicted) and $\mathcal{B}$ (ground truth),

Intersection over Union (Jaccard index)

$$J(\mathcal{A},\mathcal{B}) = \frac{|\mathcal{A}\cap\mathcal{B}|}{|\mathcal{A}\cup\mathcal{B}|} = \frac{|\mathcal{A}\cap\mathcal{B}|}{|\mathcal{A}|+|\mathcal{B}|-|\mathcal{A}\cap\mathcal{B}|}.$$

IoU is 1 for a perfect overlap and 0 for disjoint sets. The *mean IoU* (mIoU) averages the per-class scores, $J = \frac{1}{K}\sum_{k=1}^{K} J_k(\mathcal{A}_k,\mathcal{B}_k)$, where $\mathcal{A}_k,\mathcal{B}_k$ are the predicted and true pixels of class k.

Why IoU and not accuracy? IoU punishes both kinds of mistake at once: predicting too few pixels (you lose intersection) *and* predicting too many (you inflate the union). It is also insensitive to class imbalance, which is exactly why people report the per-class mean. We will meet IoU again in the next chapter as the matching criterion for bounding boxes — it is the workhorse overlap measure of all of computer vision.

2.1.1 The output of the network

The simplest possible segmentation head is almost embarrassingly direct: make the last layer a convolution with *as many feature maps as there are classes*, apply a **softmax across the channel (class) dimension at every spatial location**, and train with a per-pixel **cross-**

entropy loss. Each pixel is then just a tiny classification problem, and the loss is the sum over all pixels. Everything in this chapter is a variation on how to feed this head a feature map that is both *high resolution* and *semantically rich* — two goals that pull in opposite directions.

2.2 The building blocks

2.2.1 Recovering resolution: upsampling and unpooling

A classification backbone downsamples aggressively, so the deep feature map is tiny (e.g. 7×7). To produce a full-size label map we have to go back up. The cheap, non-learned ways to do this are collectively called *unpooling*:

- **Nearest neighbor** — copy each value into its 2×2 block.
- **"Bed of nails"** — place the value in one corner and zero-fill the rest.
- **Bilinear interpolation** — smoothly interpolate between values (the usual default).
- **Max-unpooling with switch variables** — remember *where* each max came from during pooling, and put the value back exactly there on the way up. This pairs an unpooling layer with a specific pooling layer and keeps fine spatial detail aligned.

2.2.2 Learnable upsampling: transposed convolutions

Fixed interpolation is fine, but we would rather *learn* how to upsample. That is what a **transposed convolution** does (you will also see it called "deconvolution" or "fractionally strided convolution"). Intuitively, an ordinary stride-2 convolution moves the kernel two output pixels for every one input pixel and shrinks the map; a transposed convolution does the reverse — it moves *two output pixels per one input pixel* and grows the map. Each input value acts as a weight that "stamps" a copy of the kernel onto the output, and overlapping stamps are summed.

Intuition & Mental Model

Why "transposed"? Write an ordinary convolution as a matrix–vector product $\mathbf{z} = \mathbf{W}\mathbf{x}$, where $\mathbf{W}$ is a big sparse (Toeplitz) matrix that maps, say, $\mathbb{R}^{16} \to \mathbb{R}^4$. The kernel weights appear, shifted, along its rows. Upsampling is then just multiplying by the *transpose*: $\mathbf{W}^\top$ maps $\mathbb{R}^4 \to \mathbb{R}^{16}$. Same weights, transposed connectivity — hence the name. It is a real, learnable layer, not a fixed interpolation.

2.2.3 The receptive field, and why segmentation cares

The **receptive field** of a neuron is the patch of input pixels that can possibly influence it. In classification we barely think about it; in segmentation it is central, because to label a pixel correctly the network often needs to "see" a large surrounding region (is this gray blob road or sky?). Along one spatial dimension it grows as

Receptive-field growth

$$v^L = 1 + \sum_{l=1}^{L} \left((R^{[l]} - 1) \prod_{m=1}^{l-1} S^{[m]} \right),$$

where $R^{[l]}$ is the kernel size at layer l and $S^{[m]}$ the stride at layer m. A kernel larger than 1 adds neighbourhood; crucially, that addition is *amplified by the product of all earlier strides*.

This is the tension in a nutshell. Striding/pooling is the cheapest way to grow the receptive field fast, but it also destroys the resolution we are trying to output. So either we downsample and then fight to upsample again (encoder–decoder designs), or we find a way to enlarge the receptive field *without* downsampling at all.

2.2.4 Dilated (atrous) convolutions

The second option is the **dilated** (a.k.a. *atrous*, "with holes") convolution. It spreads the kernel taps apart by a dilation factor l so they cover a wider area while still using the same number of weights:

$$(h *_l k)(a, b) = \sum_i \sum_j h(a + l\, i,\ b + l\, j)\, k(i, j).$$

With $l = 1$ this is the ordinary convolution. The receptive-field formula picks up the dilation factor as another multiplier,

$$v^L = 1 + \sum_{l=1}^{L} \left((R^{[l]} - 1)\, D^{[l]} \prod_{m=1}^{l-1} S^{[m]} \right).$$

Stacking dilated convolutions with rates $1, 2, 4, 8, \ldots$ makes the receptive field grow *exponentially* while the feature-map resolution stays put.

Intuition & Mental Model

Dilation buys you a big receptive field for free, in resolution terms: no pooling, no information thrown away. The price is paid in compute — because you never downsample, every later layer still operates on a large feature map, so there are many more neurons to evaluate. It is a resolution-for-compute trade, the mirror image of pooling.

2.3 Architectures

2.3.1 Fully Convolutional Networks (FCN, Long et al., 2015)

The foundational idea: take a classification net (AlexNet, VGG, GoogLeNet) and **"convolutionalize"** it. A fully-connected layer that maps a $7 \times 7 \times 512$ block to 4096 units is mathematically identical to a convolution with a 7×7 kernel and 4096 output channels — a 1×1-output convolution on that input. Once you see FC layers as convolutions, you can run the whole network on a *larger* image and, instead of one classification vector, you get a coarse spatial grid of class scores — a low-resolution heatmap ("tabby cat" becomes a tabby-cat heatmap).

To turn that coarse heatmap back into a full-size label map, FCN uses **transposed convolutions**, and it adds **skip connections** that pull in higher-resolution feature maps from earlier layers, so fine detail is not lost. Its main weakness is a *small effective receptive field*, which shows up as blocky, context-confused predictions (a bus is partly labelled boat, train, car).

2.3.2 Deconvolution Network (Noh et al., 2015)

DeconvNet takes the symmetry seriously and dedicates *equal compute to going back up*. It is the classic **encoder–decoder "hourglass"**: a VGG-16-style encoder squeezes the image down to a bottleneck, and a mirror-image decoder built from *unpooling* and transposed-convolution layers blows it back up. The unpooling steps use **switch variables** — the remembered max locations from the matching pooling layer — so spatial structure is restored faithfully.

2.3.3 U-Net (Ronneberger et al., 2015)

U-Net is the encoder–decoder idea *plus* skip connections, and it is still the default for biomedical image segmentation. At every level of the decoder, the corresponding encoder feature map is **concatenated** onto the upsampled feature map before the next convolutions. This hands the decoder both the high-level "what" (from the bottleneck) and the high-resolution "where" (from the skip), so it can paint sharp boundaries. The architecture is cleanly symmetric — 2×2 down/up-sampling steps, each followed by two 3×3 convolutions — and the paper introduced a **weighted loss** that up-weights the thin separating membranes between touching cells, where mistakes matter most.

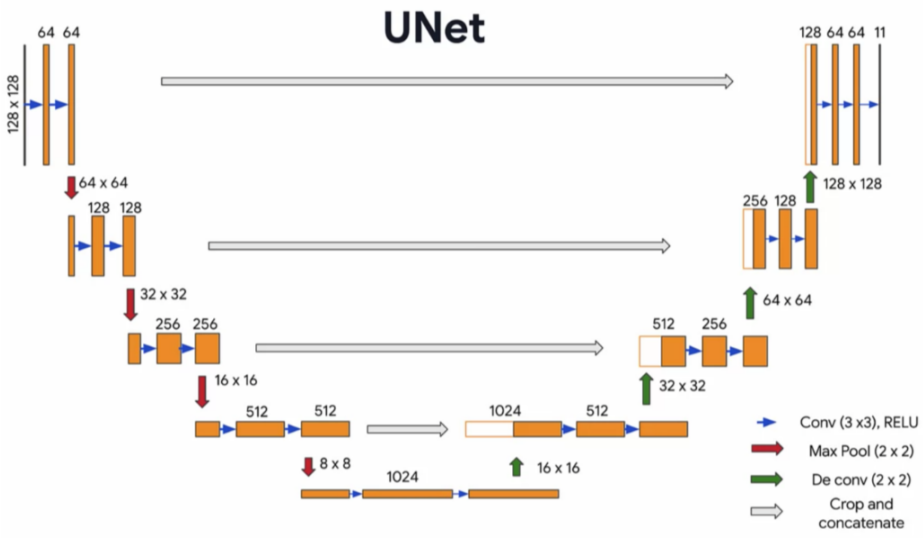

Figure 2.1: U-Net: a symmetric contracting encoder and expanding decoder in a "U" shape. Skip connections (horizontal arrows) concatenate each encoder feature map onto the matching decoder level, handing the decoder both the high-level "what" and the high-resolution "where".

2.3.4 Context aggregation by dilated convolutions (Yu & Koltun, 2016)

This is dilated convolution put to work as a dedicated **context module** bolted onto a segmentation front-end. A short stack of 3×3 convolutions with dilation $1, 1, 2, 4, 8, 16, \ldots$ aggregates multi-scale context and pushes the receptive field up to 67×67 *without any downsampling and without rescaling the image*, which measurably improves accuracy.

2.3.5 PSPNet — Pyramid Scene Parsing (Zhao et al., 2017)

PSPNet's insight is that a *single* global context vector helps, so *several context regions at different scales* should help more. Its **pyramid pooling module** pools the feature map at multiple grid

resolutions (e.g. 1×1, 2×2, 3×3, 6×6), processes each with a 1×1 convolution, upsamples them all back, and concatenates. The final classifier therefore sees context summarised at many scales at once. It won the 2017 ImageNet scene-parsing challenge and set new state of the art on Pascal VOC and Cityscapes.

2.3.6 The DeepLab family (Chen et al., 2017–2019)

DeepLab is, roughly, Google's segmentation counterpart to the Inception line in classification, and it weaves the previous ideas together.

- **DeepLab V2** introduces **Atrous Spatial Pyramid Pooling (ASPP)**: spatial pyramid pooling, but with parallel *atrous* convolutions at several rates (e.g. $6, 12, 18, 24$) instead of pooling, so multi-scale context is gathered while resolution is preserved.
- **DeepLab V3** uses atrous convolutions *inside the encoder* to keep the output stride small, and combines ASPP with a global image-pooling branch.
- **DeepLab V3+** adds an **efficient decoder** on top, getting the best of both worlds: an atrous, context-rich encoder *and* a light decoder that recovers sharp boundaries.
- **Auto-DeepLab** (2019) hands the architecture itself over to **Neural Architecture Search**: start from a densely connected super-net, learn the relative importance of each connection (and when to down/up-sample), and read off a good path. It matches DeepLab V3+ with about half the parameters and *without* ImageNet pre-training.

2.3.7 Foundation model: Segment Anything (SAM, 2023)

SAM reframes segmentation as a **promptable** task: given an image and a prompt — a point, a box, a rough mask, or even text — return a valid mask. The architecture is three parts: an **image encoder** (an MAE-pretrained vision transformer) that is run once per image, a lightweight **prompt encoder** (borrowed from CLIP, which is what lets it accept text), and a small **mask decoder** (a standard transformer decoder). It is trained with a combination of **focal loss and dice loss**. The hard part, and the real contribution, was the **data engine**: a model-in-the-loop annotation process that bootstrapped *SA-1B*, over one billion masks on eleven million images. The payoff is zero-shot transfer — SAM segments objects it was never explicitly trained on, steered purely by the prompt.

2.4 The through-line

Every architecture in this chapter is answering the same three demands at once: *fine local detail* (to get boundaries right), *strong high-level features* (to know what each region is), and *wide regional context* (to disambiguate using surroundings). Downsampling is needed to grow the receptive field cheaply and to keep compute down; upsampling (interpolation, unpooling, transposed convolutions) is needed to get back to full resolution; skip connections, dilated convolutions, and pyramid/ASPP modules are the tricks that let a network satisfy all three demands without one of them killing another.

3

Object Detection and Localization

This chapter is about putting boxes around things. The progress here over a short window was dramatic: detection accuracy (mean Average Precision) roughly tripled between 2012 and 2017, from ≈ 5 for the best pre-deep-learning method to 46 for Mask R-CNN — and, strikingly, it *also* tripled *within* the deep-learning era as architectures improved.

A note on notation

The loss functions in this chapter get crowded, so the lecture fixes a convention. Box coordinates use the upright letters $\mathsf{x}, \mathsf{y}, \mathsf{w}, \mathsf{h}$ (position, width, height) — deliberately distinct from the input $\mathbf{x}$, the label $\mathbf{y}$, weights $\mathbf{w}$, and hidden state $\mathbf{h}$. A hat denotes a prediction: $(\hat{y}_{\mathsf{x}}, \hat{y}_{\mathsf{y}}, \hat{y}_{\mathsf{w}}, \hat{y}_{\mathsf{h}})$ is a predicted box versus the label $(y_{\mathsf{x}}, y_{\mathsf{y}}, y_{\mathsf{w}}, y_{\mathsf{h}})$. A superscript i picks out region i of the image, and a subscript k picks out a class. The worst-case full quantity $\hat{y}^{i}_{k,\mathsf{x}}$ reads "predicted x-coordinate, class k, region i".

3.1 From localization to detection

Single-object localization is the easy case and the right place to start. There is exactly one object, and we want both its class and its box. The trick is to recognise that finding the box is just *regression*: take an ImageNet-pretrained CNN backbone and attach *two heads* to its final feature map. The **classification head** (often frozen) predicts the class scores; the **regression head** (fine-tuned) predicts the four box coordinates $(\hat{y}_{\mathsf{x}}, \hat{y}_{\mathsf{y}}, \hat{y}_{\mathsf{w}}, \hat{y}_{\mathsf{h}})$. Train the regression head against the ground-truth box and you are done.

Object detection is where it gets hard, for one structural reason: the number of objects is *not known in advance*. You cannot have a fixed-size output of "four coordinates" when an image might contain one sheep or fifteen. Naively, you would have to run a classifier over every possible sub-window at every scale — hopelessly expensive. The whole chapter is the story of doing this efficiently.

3.1.1 Measuring a detector

A detection is counted correct when its box overlaps the ground truth well enough, the standard threshold being IoU > 0.5. From there we count: **true positives** (correct detections, IoU > 0.5), **false positives** (spurious or badly-placed boxes), and **false negatives** (objects we missed). These give precision and recall,

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}.$$

Now sweep the confidence threshold from high to low: each setting gives a (R, P) point, tracing a precision–recall curve. **Average Precision** (AP) summarises that curve into one number; the classic definition is the 11-point interpolated average,

Average Precision (11-point interpolation)

$$\mathrm{AP} = \frac{1}{11} \sum_{R \in \{0,\, 0.1,\, \ldots,\, 1.0\}} \max_{R' \geq R} P(R').$$

For each recall level we take the *best precision achievable at that recall or higher*, then average over the eleven levels. The **mean AP (mAP)** averages AP over all classes — this is *the* headline detection metric.

3.2 The two-stage framework

Most accurate detectors share a common skeleton, worth holding in your head before the specific models:

1. **Image-level computation.** Process the whole image once, producing a feature map $\mathbf{h} = h(\mathbf{x})$ (and, in modern versions, a set of region proposals).
2. **Region proposals.** Some mechanism suggests a few hundred-to-thousand candidate boxes likely to contain objects.
3. **Region-level computation.** For each proposed region, extract its features $\boldsymbol{r} = f(\mathbf{h}, r)$, transform them to a fixed size, and feed them to one or more **heads** (classification, box regression, possibly a mask).

The history of two-stage detectors is essentially the history of making steps 1–3 cheaper and more shared. Watch what migrates from the slow per-region path into the cheap run-once-per-image path.

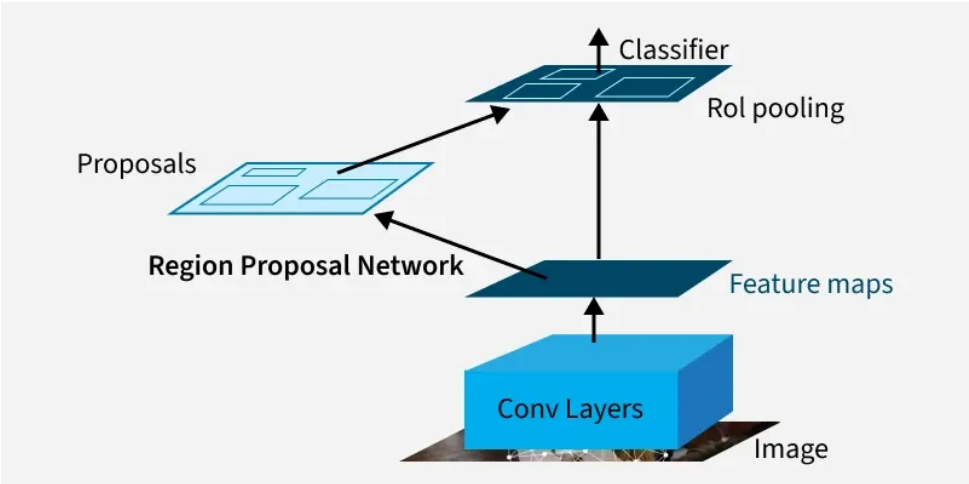

Figure 3.1: Faster R-CNN, the canonical two-stage detector: a shared backbone produces a feature map, a Region Proposal Network suggests candidate boxes, and RoI pooling feeds each proposal to classification and box-regression heads. The proposal step has migrated into the run-once-per-image path.

3.2.1 Overfeat (2013)

The earliest deep approach. It uses an AlexNet backbone, run at **multiple scales and multiple strides**, treating the convolutionalized network as a dense sliding-window detector that emits spatial feature maps. A regression layer predicts box coordinates, and a **greedy merging** algorithm fuses the hundreds of resulting boxes into final detections. It won the ImageNet localization challenge.

3.2.2 R-CNN (2014)

R-CNN splits the work cleanly into image-level and region-level stages. At image level, **selective search** — a heuristic, non-learned algorithm — proposes ∼2000 candidate regions by hierarchically grouping pixels that look alike (similar colour, texture, . . .). At region level, each proposal is **cropped and warped** to a fixed size, pushed through **AlexNet** to get features, and then classified by an **SVM**, with a separate **linear regression** refining the box offsets.

Why R-CNN is slow

The CNN is run *from scratch on every one of the ∼2000 cropped regions*. There is essentially no computation shared between overlapping proposals, so a single image takes many seconds. Every later model is, in part, a fix for this.

3.2.3 SPPNet (2014)

The key realisation: we should run the convolutional layers *once on the whole image* and only do per-region work afterwards. The obstacle is that the fully-connected head needs a *fixed-size* input, while regions have arbitrary sizes. **Spatial Pyramid Pooling (SPP)** solves this: it splits any region into a fixed number of bins (at several pyramid levels) and pools within each bin, so *any* input size yields the *same* output length. R-CNN with SPP instead of crop-and-warp is dramatically faster because the expensive convolutions are shared.

3.2.4 Fast R-CNN (2015)

Fast R-CNN cleans SPPNet up into a single, almost end-to-end model. The whole image goes through a fully-convolutional backbone once. Each proposal is then mapped onto the feature map and passed through **RoI pooling** — a one-level SPP that snaps the region to the feature grid and pools it to a fixed $h \times w$ — and the result feeds a small MLP with **two heads at once**: a softmax *classifier* and a *bounding-box regressor*. Training is now a single multi-task objective; only the region proposals remain external.

3.2.5 Faster R-CNN (2015)

The last external piece was selective search, and it was the bottleneck. Faster R-CNN replaces it with a learned **Region Proposal Network (RPN)** that shares the backbone's features — so proposals come almost for free.

The RPN slides a small network over the feature map. The clever bit is **anchor boxes**: at each location it does *not* just look at the 3×3 patch under the kernel, but considers k pre-defined anchor boxes of various sizes and aspect ratios centred there. For each anchor it predicts an **"objectness" score** (is there an object?) and a **transformation** of the anchor toward a tight box. With k anchors per location the head emits $2k$ scores and $4k$ coordinates.

Intuition & Mental Model

Anchors decouple "where to look" from the kernel size. A small 3×3 kernel can propose a large box, because it is not classifying its own little receptive field — it is predicting how to *stretch a pre-attached anchor* into the real object. That is what lets one cheap sliding network propose objects across a wide range of scales and shapes.

RPN and the detection heads share the backbone, and the lecture notes the practical recipe: a **4-step alternating training** (pre-train each, then jointly fine-tune), with SGD at learning rate ≈ 0.003 later dropped.

3.2.6 Mask R-CNN (2017): on to instance segmentation

Mask R-CNN adds a third head — a small **FCN that predicts a binary segmentation mask** for each region — on top of Faster R-CNN, giving *instance* segmentation (per-object masks, not just per-class pixels). Two details matter:

- **RoIAlign** replaces RoI pooling. RoI pooling *snaps* the region to the integer feature grid, and those roundings, harmless for a coarse box, blur a pixel-accurate mask. RoIAlign skips the snapping and samples the feature map with **bilinear interpolation** at exact locations, keeping things properly aligned.
- **Per-class sigmoid masks.** The mask branch outputs one mask per class with *independent sigmoids* (no softmax across classes), and the loss is back-propagated *only* from the mask of the ground-truth class. Decoupling mask prediction from classification this way works better than forcing the classes to compete.

3.3 One-stage detectors

Two-stage detectors are accurate but pay for the propose-then-classify pipeline. **One-stage** detectors throw out the proposal stage and predict everything in a single forward pass — much faster, historically a bit less accurate.

3.3.1 YOLO — You Only Look Once (2016)

YOLO lays an $S \times S$ grid over the image (e.g. 7×7) and makes *every grid cell* responsible for predicting objects centred in it. Each cell outputs E bounding boxes (each with coordinates plus a confidence) and a K-vector of class probabilities, so the whole network produces one fixed-size tensor.

YOLO output tensor

Each grid cell predicts E boxes ($\mathsf{x}, \mathsf{y}, \mathsf{w}, \mathsf{h}$ plus a confidence c each) and K class scores, so the output has shape

$$S \times S \times (5E + K).$$

For $S = 7$, $E = 2$, $K = 20$ this is $7 \times 7 \times 30$, i.e. $\mathbf{y} \in \mathbb{R}^{7 \times 7 \times 30}$.

The loss is a sum of several squared-error terms, each switched on by an indicator $\mathbb{1}^{\text{obj}}$ for cells that actually contain an object (and $\mathbb{1}^{\text{no obj}}$ for those that do not), weighted by coefficients

$\lambda_1, \ldots, \lambda_5$:

$$
\begin{aligned}
L(\mathbf{y}, \hat{\mathbf{y}}) = \lambda_1 & \sum_{s_1,s_2} \sum_{e} \mathbb{1}^{\text{obj}}_{s_1,s_2} \Big[(\hat{y}_{\mathsf{x}} - y_{\mathsf{x}})^2 + (\hat{y}_{\mathsf{y}} - y_{\mathsf{y}})^2 \Big] \\
& + \lambda_2 \sum_{s_1,s_2} \sum_{e} \mathbb{1}^{\text{obj}}_{s_1,s_2} \Big[(\sqrt{\hat{y}_{\mathsf{w}}} - \sqrt{y_{\mathsf{w}}})^2 + (\sqrt{\hat{y}_{\mathsf{h}}} - \sqrt{y_{\mathsf{h}}})^2 \Big] \\
& + \lambda_3 \sum_{s_1,s_2} \sum_{e} \mathbb{1}^{\text{obj}}_{s_1,s_2} (\hat{y}_{\mathsf{c}} - y_{\mathsf{c}})^2 + \lambda_4 \sum_{s_1,s_2} \sum_{e} \mathbb{1}^{\text{no obj}}_{s_1,s_2} (\hat{y}_{\mathsf{c}} - y_{\mathsf{c}})^2 \\
& + \lambda_5 \sum_{s_1,s_2} \mathbb{1}^{\text{obj}}_{s_1,s_2} \sum_{k=1}^{K} (\hat{y}_k - y_k)^2 .
\end{aligned}
$$

The λ's are there to *balance* the terms: localization versus confidence, and especially object versus the overwhelmingly more numerous no-object cells. Note the $\sqrt{}$ on width and height — it makes a fixed error on a small box count more than the same error on a large box. YOLO is fast enough for real-time detection.

3.3.2 SSD — Single Shot MultiBox Detector (2016)

SSD is one-stage like YOLO, but it predicts from **feature maps at several scales** rather than a single grid. Shallow, high-resolution maps catch small objects; deep, coarse maps catch large ones. Re-using intermediate representations this way improves multi-scale detection at one-stage speed.

3.3.3 Cleaning up: Non-Maximum Suppression

Whatever the detector, it spits out many overlapping boxes for the same object. **Non-Maximum Suppression (NMS)** is the final filter: keep the box with the highest confidence, compute its IoU against the rest, and discard any that overlap it too much; repeat. What was a thicket of red boxes becomes one clean box per object.

3.4 How they compare, and what came next

The trade-off is consistent: **two-stage methods are accurate but slow**; **one-stage methods are fast but less accurate**. Bigger backbones (ResNet, Inception-ResNet) raise accuracy, with diminishing returns as you spend more GPU time. After Mask R-CNN the field moved toward detection in *point clouds* (Point R-CNN — 3D detection from LIDAR is a research focus at JKU, where localizing oriented 3D boxes from unordered points is much harder than 2D), detection in *video*, efficiency improvements, and swapping in **ViT backbones** under the same Mask R-CNN framework. Two threads the lecture flags but does not detail are region-based fully-convolutional networks (R-FCN) and feature pyramid networks (FPN).

4

Autoencoders

With autoencoders we leave supervised vision behind and start the *generative and unsupervised* half of the course. It helps to see the map first. The methods in this part of the course split into two families. On one side sit the genuine **generative models**, which try to capture the data distribution $p(\mathbf{x})$ so you can sample new data from it; these split further into *explicit-density* models — *tractable* ones like normalizing flows and autoregressive flows, and *approximate* ones like VAEs — and *implicit-density* models like GANs. On the other side sit the **density-free** methods — and that is where the autoencoder lives.

Intuition & Mental Model

This placement is the single most important thing to understand about a plain autoencoder: it is *density-free*. It never models $p(\mathbf{x})$ and it is not, by itself, a generative model. It just learns to *compress and reconstruct*. Everything that feels disappointing about an autoencoder when you try to generate with it, and everything the VAE adds in the next chapter, follows from this one fact.

4.1 The basic idea

An autoencoder is a network trained to copy its input to its output — which sounds pointless until you force the copy to pass through a *bottleneck*. It has two halves. The **encoder** f maps the input to a low-dimensional *code* (or *latent representation*) $\mathbf{z} = f(\mathbf{x})$, and the **decoder** g maps that code back to a reconstruction $\hat{\mathbf{x}} = g(\mathbf{z})$. Training is unsupervised — no labels, the input is its own target — and the objective is simply to make the reconstruction resemble the input, typically with a squared-error **reconstruction loss**

$$\mathcal{L}(\boldsymbol{\theta}) = \left\| \mathbf{x} - g\big(f(\mathbf{x})\big) \right\|^2 ,$$

averaged over the data. Because the code is much smaller than the input, the network cannot just pass the data through; it has to discover an efficient, compressed encoding that keeps whatever is needed to reconstruct. That learned code is the whole point.

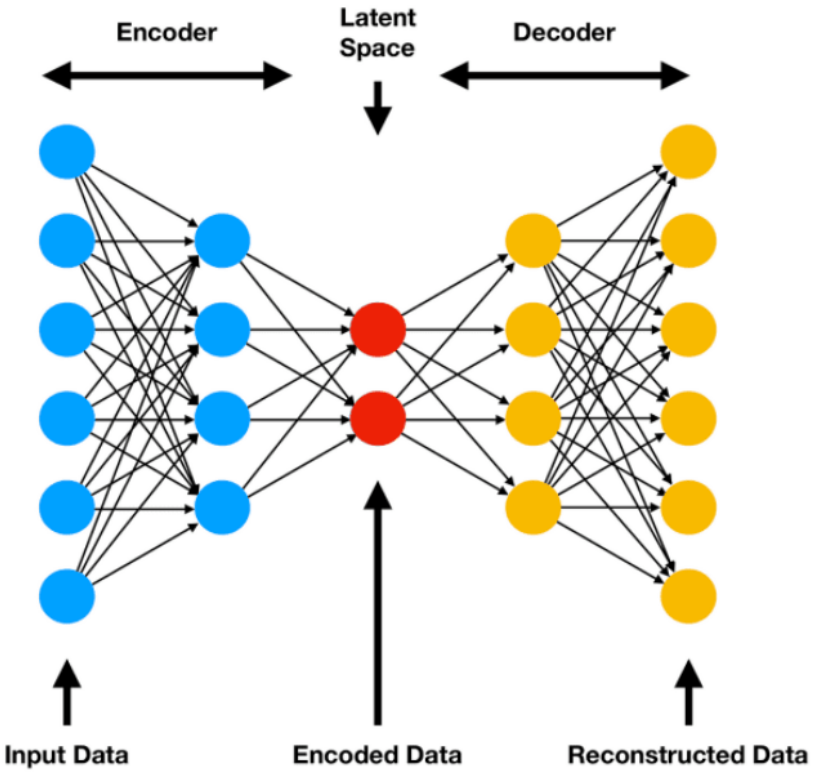

Figure 4.1: Autoencoder: an encoder f compresses the input $\mathbf{x}$ to a low-dimensional code $\mathbf{z}$ at the bottleneck, and a decoder g reconstructs $\hat{\mathbf{x}}$. Training minimises the reconstruction error between $\mathbf{x}$ and $\hat{\mathbf{x}}$.

4.2 The bottleneck is what makes it learn

Undercomplete vs. overcomplete

An **undercomplete** autoencoder has a code smaller than the input ($\dim \mathbf{z} < \dim \mathbf{x}$). The dimensionality squeeze is what forces it to learn structure. An **overcomplete** autoencoder has a code at least as large as the input; without further constraints it can learn the trivial identity ("copy everything"), which teaches it nothing.

So the lesson is that an autoencoder learns something useful only when it is *prevented* from learning the identity — either by squeezing the code (undercomplete) or by adding a regularizer (the variants below). A perfect copier is a useless autoencoder.

Intuition & Mental Model

A neat anchor point: a **linear autoencoder** with a single hidden layer and a squared-error loss learns essentially the same subspace as **PCA** — it projects onto the top principal components. Viewed this way, an autoencoder is "PCA with non-linearities": the encoder and decoder can bend the projection into curved manifolds that a linear method like PCA cannot capture.

4.3 Regularized autoencoders

If we want representations that are useful (and we want to use overcomplete codes without collapsing to the identity), we constrain *what kind* of code the network is allowed to learn. Three classic variants:

- **Denoising autoencoder (DAE).** Corrupt the input — add noise, mask out pixels — and ask the network to reconstruct the *clean* original. To undo corruption it has to understand the underlying structure of the data rather than copy surface detail, so it learns robust features. This is also the conceptual ancestor of masked-image-modelling and diffusion ideas later in the course.
- **Sparse autoencoder.** Add a penalty that pushes most hidden activations toward zero, so each input is explained by only a few active units. The constraint here is on *how many units may fire*, not on how many units exist, which lets you use a large (even overcomplete) code while still learning meaningful, selective features.
- **Contractive autoencoder (CAE).** Penalize the norm of the encoder's Jacobian, $\|\partial f/\partial \mathbf{x}\|^2$. This makes the code *insensitive* to small input perturbations, so the representation varies only along the directions that actually matter (the data manifold) and stays flat elsewhere.

4.4 Why a plain autoencoder cannot generate

It is tempting to think we can generate new data by picking a random code $\mathbf{z}$ and decoding it. With a plain autoencoder this fails, and the reason is exactly the density-free nature flagged at the start: **nothing organises the latent space**. The encoder is free to scatter training points wherever it likes, leaving the code space full of "holes" — regions that correspond to no real data. Sample a $\mathbf{z}$ from one of those holes, decode it, and you get garbage. The autoencoder learned to reconstruct *the points it saw*, not a smooth, fillable distribution over codes.

The gap the VAE fills

The fix, in the next chapter, is to make the latent space *probabilistic and well-behaved*: a Variational Autoencoder forces the codes to follow a known prior (typically a standard Gaussian) so that *any* sample from that prior decodes to something plausible. That single change — putting a tractable density on the latent space — is what turns the encoder–decoder idea from a density-free compressor into a real generative model.

4.5 What autoencoders are good for

Even though it is not a generator, the plain autoencoder is genuinely useful:

- **Dimensionality reduction** — a non-linear, learnable generalisation of PCA, for visualisation or as a compact feature.
- **Representation learning / pre-training** — learn features from abundant unlabelled data, then fine-tune on a small labelled set.
- **Denoising** — the DAE is directly a denoiser.
- **Anomaly detection** — train only on "normal" data; anything the autoencoder reconstructs *badly* (high reconstruction error) is flagged as anomalous, because it lies off the manifold the model learned.

5

Variational Autoencoders

The previous chapter ended on a complaint: a plain autoencoder learns to reconstruct the points it saw, but its latent space is full of holes, so you cannot sample from it. The **Variational Autoencoder** (VAE, Kingma & Welling, 2014) is the fix. It keeps the encoder–decoder shape but turns the whole thing into a proper probabilistic generative model, which lands it in the *explicit, approximate-density* branch of the taxonomy — right next to diffusion models, and notably *not* in the density-free corner where the plain AE sits.

5.1 The generative story

A VAE describes how data is *born*. First draw a latent code from a fixed, simple prior — a standard Gaussian, $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ — then push it through a **decoder** $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$ (a neural network) to produce data. Sampling is then trivial *by construction*: draw $\mathbf{z}$ from the Gaussian, decode it, done. This is exactly what the plain AE could not promise, because here the prior *defines* which codes are valid.

The catch is training. The quantity we would like to maximise is the marginal likelihood of the data,

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, \mathrm{d}\mathbf{z},$$

but this integral runs over all of latent space and is intractable. Worse, the posterior $p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})$ — which code produced this data point? — is intractable too, so we cannot train by straightforward maximum likelihood.

5.2 The encoder as approximate inference

The VAE's trick is to introduce an **encoder** $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$ that *approximates* the intractable posterior. In practice the encoder maps each input to the parameters of a Gaussian over codes,

$$q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z}\,;\, \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}),\, \mathrm{diag}(\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x}))\big),$$

i.e. it outputs a mean $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x})$ and a (log-)variance $\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x})$ per input. So instead of mapping $\mathbf{x}$ to a single point in latent space (as a plain AE does), the VAE maps it to a little *cloud* — a distribution over plausible codes.

5.3 The ELBO

We cannot maximise $\log p_{\boldsymbol{\theta}}(\mathbf{x})$ directly, so we maximise a tractable lower bound on it: the **Evidence Lower BOund** (ELBO).

Evidence Lower Bound

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})]}_{\text{reconstruction}} \;-\; \underbrace{D_{\mathrm{KL}}(q_\phi(\mathbf{z}\,|\,\mathbf{x})\,\|\,p(\mathbf{z}))}_{\text{regularizer}} \;=\; \text{ELBO}.$$

The gap between $\log p_{\boldsymbol{\theta}}(\mathbf{x})$ and the ELBO is exactly $D_{\mathrm{KL}}(q_\phi(\mathbf{z}\,|\,\mathbf{x})\,\|\,p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x}))$ — how far the encoder's approximation sits from the true posterior. Since a KL is never negative, the ELBO is a genuine lower bound, and tightening it both fits the data and improves the approximate posterior.

The two terms pull in complementary directions, and reading them is the whole intuition of the VAE:

- The **reconstruction term** wants codes that let the decoder rebuild the input — exactly the old autoencoder objective.
- The **KL regularizer** wants every input's code-cloud to look like the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$. This is the part the plain AE was missing: it forces the encoder to *pack all the codes into the same standard Gaussian*, leaving no holes. That is precisely what makes sampling work afterwards.

Intuition & Mental Model

The KL term is the cure for the plain AE's broken latent space. Without it, the encoder scatters codes wherever it likes. With it, the codes are gently squeezed to overlap the prior, so a random draw from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ at generation time lands somewhere the decoder actually understands. You pay for this with some reconstruction fidelity — hence the characteristic VAE trade-off.

When the prior is $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and the encoder is Gaussian, the KL term has a clean closed form (no sampling needed),

$$D_{\mathrm{KL}}(q_\phi(\mathbf{z}\,|\,\mathbf{x})\,\|\,p(\mathbf{z})) = \tfrac{1}{2}\sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2),$$

which is one reason Gaussians are chosen on both sides.

5.4 The reparameterization trick

There is one obstacle to training the ELBO with backpropagation: the reconstruction term requires *sampling* $\mathbf{z}$ from $q_\phi(\mathbf{z}\,|\,\mathbf{x})$, and you cannot backpropagate through a random draw. The **reparameterization trick** moves the randomness out of the way:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Now the noise $\boldsymbol{\epsilon}$ is an *external input*, and the path from the encoder parameters ϕ through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ to $\mathbf{z}$ is fully deterministic and differentiable. Gradients flow straight through, and the whole VAE trains end-to-end with ordinary SGD.

5.5 Putting it together

Training is therefore one tidy loop: the encoder produces $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$; we sample $\mathbf{z}$ via reparameterization; the decoder reconstructs $\hat{\mathbf{x}}$; and we descend on $-\text{ELBO} = (\text{reconstruction loss}) + (\text{KL})$. The likelihood $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$ is chosen to match the data — a Gaussian for real-valued pixels (giving a squared-error reconstruction loss) or a Bernoulli for binary data (giving cross-entropy).

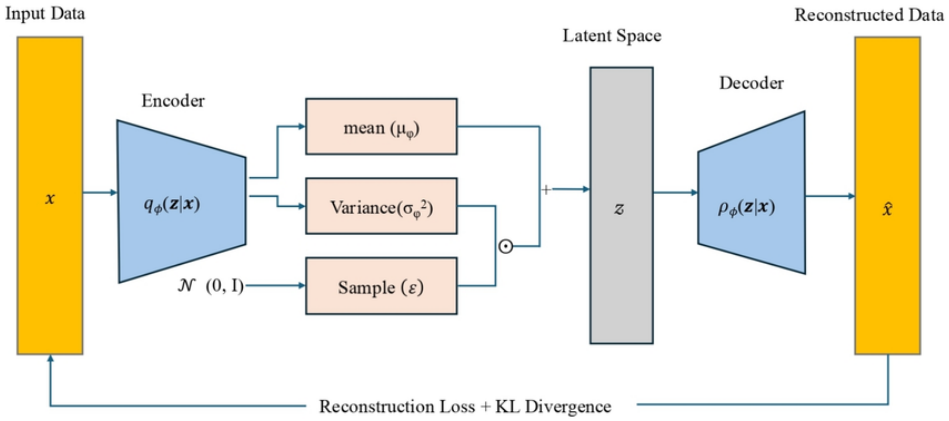

Figure 5.1: VAE: the encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ outputs a mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma}^2$; a latent $\mathbf{z}$ is drawn by the reparameterization trick $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$; and the decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ reconstructs the input. The loss is reconstruction plus a KL term pulling q_ϕ toward the prior.

Two things that go wrong

Blurry samples. VAEs are famous for producing softer, blurrier images than GANs. A pixel-wise Gaussian likelihood rewards predicting the *average* of all plausible reconstructions, and an average of sharp images is a blurry one.

Posterior collapse. If the decoder is powerful enough to model the data on its own, the optimiser can drive the KL term to zero by making $q_\phi(\mathbf{z} \mid \mathbf{x}) \approx p(\mathbf{z})$ for every input — the latent code is ignored. Tricks like KL warm-up or β**-VAE** (which scales the KL by a factor β to trade off reconstruction against latent structure) are used to manage this.

The VAE is the first true deep generative model in this course: a single objective (the ELBO), a single trick (reparameterization), and a latent space you can actually sample from. Keep the three-term reading in mind — reconstruction versus a KL-to-prior regularizer — because the diffusion models in Chapter 8 are, at heart, a deep stack of exactly this idea.

6

GANs and W-GANs

VAEs and flows model the data density explicitly. **Generative Adversarial Networks** (GANs, Goodfellow et al., 2014) take the opposite stance: they sit in the *implicit-density* branch, meaning they never write down $p(\mathbf{x})$ at all. A GAN only knows how to *produce samples* — and it learns to produce good ones by being put in a competition.

6.1 The adversarial game

The whole idea is two networks playing against each other. The **generator** G takes random noise $\mathbf{z} \sim p_{\mathbf{z}}$ from a latent space and turns it into a fake sample $G(\mathbf{z})$. The **discriminator** D is a classifier that looks at a sample — real or fake — and tries to tell which it is. The generator's job is to fool the discriminator; the discriminator's job is to not be fooled.

Intuition & Mental Model

The classic analogy: G is a counterfeiter printing fake banknotes, D is the police trying to spot them. As the police get better, the counterfeiter is forced to make better fakes, which forces the police to get sharper still. At the equilibrium of this arms race the fakes are indistinguishable from real money — i.e. the generator's distribution matches the data.

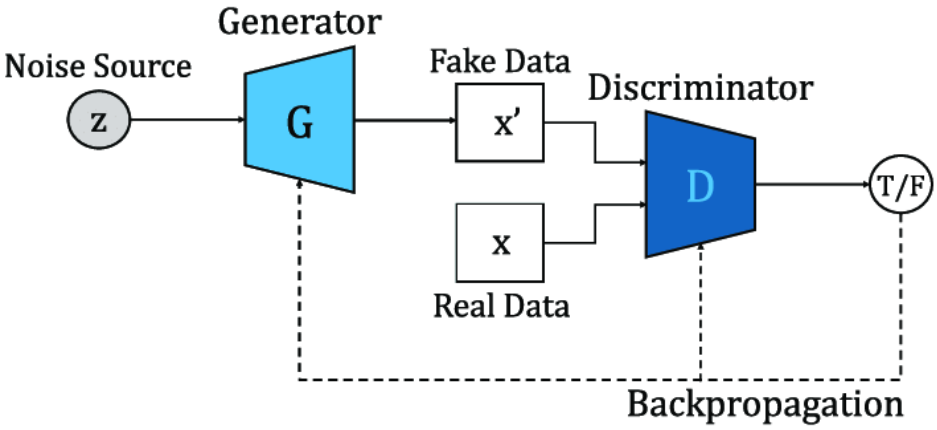

Figure 6.1: GAN: the generator G maps a noise vector $\mathbf{z} \sim p_{\mathbf{z}}$ to a fake sample $G(\mathbf{z})$; the discriminator D receives either a real sample or a fake one and outputs the probability that it is real. The two networks are trained against each other.

6.2 The objective

Both networks are trained against a single value function in a *minimax* game — the discriminator maximises it, the generator minimises it.

GAN minimax objective

$$\min_G \max_D \; \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_\mathbf{z}}[\log(1 - D(G(\mathbf{z})))],$$

where p_r is the real-data distribution and p_g the generator's. The discriminator wants $D(\mathbf{x}) \to 1$ on real data and $D(G(\mathbf{z})) \to 0$ on fakes; the generator wants the opposite. For a fixed G the optimal discriminator is $D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x})+p_g(\mathbf{x})}$, and plugging it back in shows the generator is then minimising the *Jensen–Shannon divergence* between p_r and p_g.

In practice training alternates two gradient steps: ascend on the discriminator's parameters $\mathbf{w}$ and descend on the generator's parameters $\boldsymbol{\theta}$,

$$\mathbf{w} \leftarrow \mathbf{w} + b\,\nabla_\mathbf{w} \tfrac{1}{M} \textstyle\sum_i \big[\log D_\mathbf{w}(\mathbf{x}^i) + \log(1 - D_\mathbf{w}(G_{\boldsymbol{\theta}}(\mathbf{z}^i)))\big],$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - a\,\nabla_{\boldsymbol{\theta}} \tfrac{1}{M} \textstyle\sum_i \log(1 - D_\mathbf{w}(G_{\boldsymbol{\theta}}(\mathbf{z}^i))),$$

on minibatches of real samples and noise. In code, the generator usually *maximises* $\log D(G(\mathbf{z}))$ instead of minimising $\log(1 - D(G(\mathbf{z})))$ — the "non-saturating" version — because the original form has vanishing gradients exactly when the generator is losing and needs signal most.

6.3 Why GANs are hard to train

There is no likelihood to monotonically improve, just a moving target, and that makes GANs notoriously finicky:

- **Non-convergence.** Two networks chase each other; the dynamics can oscillate instead of settling.
- **Mode collapse.** The generator discovers a few outputs that reliably fool D and produces only those, ignoring whole regions of the data distribution (e.g. generating one kind of face over and over).
- **Vanishing gradients.** If D becomes too good too fast, it confidently rejects every fake, $\log(1 - D(G(\mathbf{z})))$ saturates, and the generator receives almost no gradient.

When they do work, the payoff is sharp samples: with a convolutional generator and discriminator (**DCGAN**) GANs produce noticeably crisper images than a VAE trained on the same faces, because there is no pixel-averaging likelihood blurring things out.

6.4 Wasserstein GANs

Many of the gradient problems trace back to the *divergence* a vanilla GAN optimises. When p_r and p_g have little overlap — which is the norm in high dimensions, where both live on thin manifolds — the Jensen–Shannon divergence is essentially constant (it pins at $\log 2$). A constant has zero gradient, so the generator gets no useful direction to move in. This is the same KL/JS-divergence pathology that motivated FID over older metrics, and the fix is the same: switch to a **Wasserstein** distance.

Wasserstein-1 (Earth-Mover) distance

Informally, $W(p_r, p_g)$ is the minimum "cost" of transporting probability mass to reshape p_g into p_r, where cost is mass times distance moved. Crucially it stays *finite and smoothly varying even when the two distributions do not overlap* — so it always gives the generator a meaningful gradient pointing toward the data.

Computing an Earth-Mover distance directly is intractable, but the Kantorovich–Rubinstein duality rewrites it as

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{\mathbf{x}\sim p_r}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x}\sim p_g}[f(\mathbf{x})],$$

a supremum over all **1-Lipschitz** functions f. In a WGAN we simply let a network — now called a **critic** rather than a discriminator — play the role of f. The critic outputs an unbounded real score (no sigmoid, no "real/fake probability"); it just scores how real something looks, and the generator tries to raise the critic's score on its fakes.

The one new requirement is keeping the critic 1-Lipschitz:

- The original **WGAN** enforced it crudely by *weight clipping* — clamping all critic weights into $[-c, c]$.
- **WGAN-GP** replaced clipping with a *gradient penalty* that pushes the norm of the critic's input-gradient toward 1. This is much more stable and is the standard recipe; on CelebA it reaches an excellent FID of around 3.

Intuition & Mental Model

Two practical wins make WGANs worth the trouble. First, training is far more stable and largely free of mode collapse, because the critic always supplies a usable gradient. Second — and unusually for GANs — the critic's loss *correlates with sample quality*, so for once the training curve is a meaningful progress meter rather than noise.

7

Metrics, Normalizing Flows & Density Models

This chapter does two things. First it asks a question we have been dodging: now that we can generate images, *how do we measure whether they are any good?* That leads to the FID metric. Second, it introduces the model family we have been building toward — **normalizing flows** — which, unlike VAEs and GANs, gives the data density *exactly*. Along the way it rounds out the map of generative models with autoregressive models and energy-based models.

7.1 The map of generative models

It is worth pinning down the whole landscape, because every model in this part of the course is a point on one tree:

- **Explicit density** — the model writes down $p(\mathbf{x})$.
  - *Exact / tractable density*: **autoregressive models** and **normalizing flows** — you can evaluate $p(\mathbf{x})$ exactly.
  - *Unnormalized density*: **energy-based models** — you get $p(\mathbf{x})$ up to an intractable normalizing constant.
  - *Approximate density*: **VAEs** and **diffusion models** — you optimise a lower bound (the ELBO) instead of the true likelihood.
- **Implicit density** — **GANs**: no $p(\mathbf{x})$, only a sampler.
- **Density-free** — **autoencoders**: not generative at all, just compression.

That "exact density" column is the prize: a model you can both *sample* from and *score* (evaluate the likelihood of). Flows are the headline example and the heart of this chapter.

7.2 How do you score a generator?

GANs exposed an awkward gap: because they have only an implicit density, there is *no likelihood to report*, so for a while the quality of generated images was judged by eyeballing them. The discriminator is no help as a general yardstick — it is trained against one specific generator (it is "on-policy") and does not transfer. We want an automatic metric that *agrees with human judgement* of what a realistic, varied set of images looks like. Two became standard, both built on a pretrained InceptionNet classifier.

7.2.1 Inception Score (IS)

The idea: feed each generated image through InceptionNet and look at the predicted label distribution $p_{\text{Inc}}(y \mid \mathbf{x})$.

Inception Score

$$\text{IS}(G) = \exp\left(\frac{1}{N}\sum_{n=1}^{N} D_{\text{KL}}(p_{\text{Inc}}(y \mid \mathbf{x}_n) \,\|\, p_{\text{Inc}}(y))\right), \qquad p_{\text{Inc}}(y) = \frac{1}{N}\sum_{n=1}^{N} p_{\text{Inc}}(y \mid \mathbf{x}_n),$$

where $p_{\text{Inc}}(y)$ is the marginal label distribution over all generated images. Higher is better.

The score rewards two things at once, which is easiest to see through entropy. Each per-image distribution $p_{\text{Inc}}(y \mid \mathbf{x}_n)$ should be *sharp* (low entropy) — a good image clearly depicts *one* recognizable thing. The marginal $p_{\text{Inc}}(y)$ should be *flat* (high entropy) — across many images the generator should cover many classes, i.e. be diverse. A large KL between the two means exactly "confident per image, diverse overall."

Why IS is not enough

IS is easy to game. It cannot see *intra-class* diversity, so a generator that emits one perfect dog, one perfect cat, etc. (mode collapse within each class) still scores well. It rewards outright *memorizing* training images. And it is blind to anything outside InceptionNet's classes — generate a flawless object it was never trained on and IS will be low; generate a person with two heads and it may not be penalized at all.

7.2.2 Fréchet Inception Distance (FID)

FID fixes most of this by comparing *distributions of features* rather than labels. Run both the real and the generated images through InceptionNet, take the penultimate-layer activations, and fit a Gaussian to each set. FID is then the *Wasserstein-2 distance* between those two Gaussians.

Fréchet Inception Distance

$$\text{FID} = \|\boldsymbol{m} - \boldsymbol{m}_r\|_2^2 + \text{tr}\Big(\boldsymbol{C} + \boldsymbol{C}_r - 2(\boldsymbol{C}\,\boldsymbol{C}_r)^{1/2}\Big),$$

where $(\boldsymbol{m}, \boldsymbol{C})$ and $(\boldsymbol{m}_r, \boldsymbol{C}_r)$ are the mean and covariance of the generated and real feature sets. **Lower is better** (0 = identical statistics).

Note the recurring theme: just like WGAN traded JS divergence for a Wasserstein distance, FID upgrades the KL-based IS to a Wasserstein-2 distance. FID is meaningful even for classes InceptionNet never saw, it *detects mode collapse and low variability* (a narrow generator gives a narrow covariance, inflating the distance), and it tracks human judgement well — on CelebA, samples at FID 133 look rough while FID 13 looks clean, and a WGAN-GP at FID 3 looks essentially real. A close cousin, the **Kernel Inception Distance (KID)**, uses a kernel two-sample test on the same features. Empirically FID rises smoothly as you add noise, blur, or occlusion to images, whereas IS barely reacts — another reason FID is the field's default.

7.3 Do GANs even converge?

A fair worry: with two networks influencing each other, both updated on noisy minibatches, is there any guarantee training settles down? Writing the updates as a stochastic-approximation

scheme,

$$\mathbf{w}_{n+1} = \mathbf{w}_n + b(n)(\boldsymbol{g}(\boldsymbol{\theta}_n, \mathbf{w}_n) + \boldsymbol{M}_n^{(\mathbf{w})}), \qquad \boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n + a(n)(\boldsymbol{h}(\boldsymbol{\theta}_n, \mathbf{w}_n) + \boldsymbol{M}_n^{(\boldsymbol{\theta})}),$$

(where $\boldsymbol{g}, \boldsymbol{h}$ are the full gradients and $\boldsymbol{M}$ the noise), one can lean on the theory of stochastic approximation (Borkar, 1997). The key result — the **Two Time-Scale Update Rule (TTUR)**, from Heusel et al. (2017), the same JKU group behind FID — shows that with *separate* learning rates $a(n)$ and $b(n)$ for generator and discriminator, the process *does* converge to a local Nash equilibrium under mild assumptions.

7.4 Normalizing flows

Now the main event. GANs give an implicit density and VAEs only an approximate one, so neither lets you actually evaluate $p(\mathbf{x})$. **Normalizing flows** do, exactly. The whole idea fits in one line:

$$\mathbf{x} = T(\boldsymbol{u}), \qquad \boldsymbol{u} \sim p_u(\boldsymbol{u}),$$

take a simple **base distribution** p_u (a standard Gaussian) and pass a sample through an **invertible, differentiable** transformation T (a neural network). Generating is forward, $\mathbf{x} = T(\boldsymbol{u})$; the magic is that we can also compute the exact density of any $\mathbf{x}$.

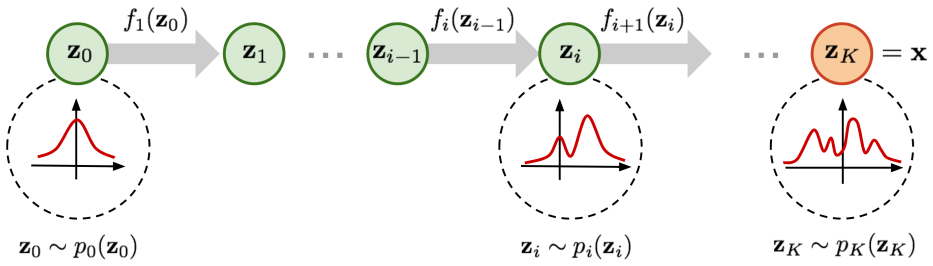

Figure 7.1: A normalizing flow gradually transforms a simple base density p_u (e.g. a standard Gaussian) into a complex data density p_x through a chain of invertible maps. Sampling runs forward ($\mathbf{x} = T(\boldsymbol{u})$); density evaluation runs backward through T^{-1} with the Jacobian-determinant correction.

7.4.1 Change of variables

The bridge is the change-of-variables formula from probability.

Change of variables

$$p_x(\mathbf{x}) = p_u(T^{-1}(\mathbf{x}))\,|\det J_{T^{-1}}(\mathbf{x})|.$$

The Jacobian-determinant factor accounts for how T locally stretches or squeezes volume: where the map expands space, density thins out, and vice versa.

Intuition & Mental Model

Picture a uniform blob of probability over a unit square. Apply a linear map $\boldsymbol{A}$ and the square becomes a parallelogram of area $|\det \boldsymbol{A}|$; since the *total* probability must stay 1, the density scales by $1/|\det \boldsymbol{A}|$. A flow does this continuously and non-linearly — it "molds" the simple base density into the complicated shape of the data, like kneading dough from a round ball into an intricate form.

A single invertible step is weak, so flows *compose* many: $T = T_K \circ \cdots \circ T_1$. Composition is friendly to both operations we need — the inverse reverses the order, and the log-determinant is just a sum,

$$\log|\det J_T(\boldsymbol{z})| = \sum_k \log|\det J_{T_k}(\boldsymbol{z}_{k-1})|.$$

The name "flow" is the trajectory of a sample as it is carried from the base space ($\boldsymbol{z}_0 = \boldsymbol{u}$) to data ($\boldsymbol{z}_K = \mathbf{x}$) through these steps. Training is plain maximum likelihood: minimising $D_{\mathrm{KL}}(p^* \,\|\, p_{\boldsymbol{\theta}})$ reduces to

$$L(\boldsymbol{\theta}) \approx -\frac{1}{N}\sum_{n=1}^{N}\Big[\log p_u(T^{-1}(\mathbf{x}_n)) + \log|\det J_{T^{-1}}(\mathbf{x}_n)|\Big],$$

i.e. make the data probable under the base distribution after pulling it back through T^{-1}, while accounting for the volume change.

7.4.2 The engineering constraint, and autoregressive flows

Since T is a neural network, the design is dominated by one demand: it must be **invertible** *and* have a **tractable Jacobian determinant**. A general $D \times D$ determinant costs $\mathcal{O}(D^3)$ — far too much. The standard escape is to make the Jacobian **triangular**, because then the determinant is just the product of its diagonal.

Autoregressive flows achieve this. Each output coordinate is transformed using only the *earlier* coordinates,

$$z_i' = \tau(z_i;\, \boldsymbol{h}_i), \qquad \boldsymbol{h}_i = c_i(\boldsymbol{z}_{<i}),$$

where the **transformer** τ is a strictly monotonic (hence invertible) function of z_i, and the **conditioner** c_i — which can be any network, since it need not be invertible — produces its parameters from the preceding coordinates. Because z_i' ignores $z_{j>i}$, the Jacobian is triangular and

$$\log|\det J| = \sum_{i=1}^{D} \log\Big|\tfrac{\partial \tau}{\partial z_i}(z_i; \boldsymbol{h}_i)\Big|.$$

The simplest choice is the **affine** transformer $\tau(z_i; \boldsymbol{h}_i) = \alpha_i z_i + \beta_i$, whose log-determinant is just $\sum_i \log|\alpha_i|$. This template covers the famous flows — **NICE**, **RealNVP**, **IAF**, **MAF**, and **Glow** — which differ mainly in how the conditioner is implemented (e.g. recurrently with an RNN/LSTM, or with a *masked* feed-forward net that zeroes out the forbidden connections).

Intuition & Mental Model

There is a satisfying theoretical guarantee here: flows are *universal*. Using the chain rule of probability and the fact that the CDF of each conditional maps it to a uniform variable, one can show a flow can represent *any* sufficiently nice distribution — even with a base as humble as a uniform on the unit cube. The practical art is doing it with transformations that are also cheap to invert and differentiate.

The standard demo is morphing a 2-D Gaussian into a two-half-moons distribution through a 9-layer RealNVP, and running it backwards to recover the Gaussian. The applications are exactly the three things an exact-density model is good for: **density estimation**, **generation/sampling**, and **inference**.

7.5 Autoregressive models

Worth a short standalone note, since autoregressive flows borrow their name. The idea is the chain rule of probability applied to generation:

$$p(\mathbf{x}) = \prod_{d=1}^{D} p(x_d \,|\, \mathbf{x}_{<d}),$$

generate one piece at a time, each conditioned on everything generated so far — *pixel-by-pixel* for images (**PixelRNN**, **PixelCNN**), or *token-by-token* for text. Each piece is typically a discrete variable drawn from a softmax. This is the family that **large language models** belong to, and it is treated in depth in the recurrent-networks material; for images, pixel-by-pixel generation is exact but slow and largely superseded by GANs and diffusion.

7.6 Energy-based models

The last branch of the tree. Rather than a normalized density, an **EBM** learns an *energy* $E_{\boldsymbol{\theta}}(\mathbf{x})$ and sets

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{\exp\!\big(-E_{\boldsymbol{\theta}}(\mathbf{x})\big)}{Z_{\boldsymbol{\theta}}}, \qquad Z_{\boldsymbol{\theta}} = \int \exp\!\big(-E_{\boldsymbol{\theta}}(\mathbf{x})\big)\, \mathrm{d}\mathbf{x}.$$

Low energy = high probability. The normalizing constant $Z_{\boldsymbol{\theta}}$ (the **partition function**) is intractable, so EBMs deliberately give up on the exact value of p and care only about the *relative ordering* of densities. Training differentiates the negative log-likelihood into a *contrastive* form — a "data gradient" pushing energy down on real samples and a "model gradient" pushing it up on samples drawn from the model (a structure that resembles the WGAN objective). Those model samples come from MCMC, in practice **Stochastic Gradient Langevin Dynamics**:

$$\mathbf{x}^{t+1} \leftarrow \mathbf{x}^{t} + \tfrac{\alpha}{2} \nabla_{\mathbf{x}} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{t}) + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \alpha \mathbf{I}),$$

where the injected noise keeps the iterate from collapsing onto a mode and instead gives unbiased samples from the distribution.

EBMs have appealing properties — *compositionality* (add or subtract energy functions to combine concepts), *no generator network* needed (any standard classifier architecture can serve as the energy), and *adaptive compute* (run the sampler longer for finer samples). The formulation shown is not widely used today (though championed by LeCun); the better-known EBMs are **Restricted Boltzmann Machines** and **Hopfield networks** (including their modern continuous variants). Keep SGLD in mind — it returns, in disguise, when we interpret diffusion models next.

8

Diffusion Probabilistic Models

Diffusion probabilistic models (DPMs) are the engine behind the modern image generators (Stable Diffusion, DALL · E, Imagen), and conceptually they are the payoff of everything in this part of the course. They live in the same *explicit, approximate-density* box as the VAE, and in fact a DPM is best read as a **very deep, hierarchical VAE** with a particularly clever twist: the encoder is not learned at all.

8.1 The main idea

Take a clean image and gradually destroy it by adding a little Gaussian noise over and over, hundreds of steps, until nothing is left but pure static. That is the **forward (diffusion) process** — and it is completely fixed, no learning involved. Now imagine running it *backwards*: starting from static and removing a little noise at each step until an image emerges. That **reverse (denoising) process** is what we train a network to do. Once it can denoise, generation is simple — sample pure noise and let the network walk it back to a clean sample.

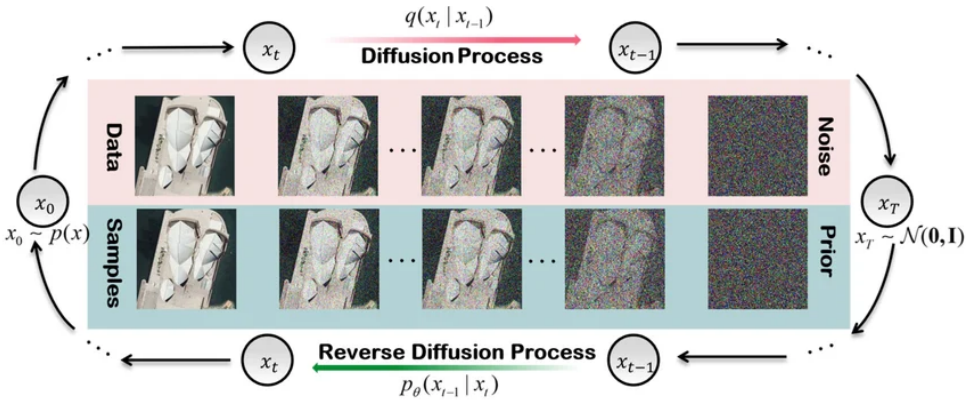

Figure 8.1: The two Markov chains of a diffusion model: the fixed forward process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ gradually adds Gaussian noise until $\mathbf{x}_T$ is pure static, and the learned reverse process $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ denoises step by step to generate a sample.

Notation

The data point is $\mathbf{x}_0 = \mathbf{x}$ and we noise it over steps $t = 1, \ldots, T$ to get a sequence $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T$, where $\mathbf{x}_T$ is essentially pure $\mathcal{N}(\mathbf{0}, \mathbf{I})$ noise. The time index is *zero-based*. The forward step is $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ (fixed); the learned reverse step is $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$.

8.2 The forward process (a fixed encoder)

Each forward step adds a touch of noise while slightly shrinking the existing signal:

$$\mathbf{x}_t = \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

which as a distribution is $q(\mathbf{x}_t \,|\, \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t\,;\, \sqrt{\alpha_t}\,\mathbf{x}_{t-1},\, (1-\alpha_t)\mathbf{I})$. This is a fixed (un-learned) *linear Gaussian* encoder, a Markov chain, whose noise schedule $\{\alpha_t\}$ is set so that the latent has the same dimension as the data and the final step is pure standard-Gaussian noise.

The beautiful part is that you do not have to simulate all t steps. Composing the Gaussians and marginalising out the intermediate states gives a **diffusion kernel** that jumps straight from the clean image to *any* noise level in one shot:

Diffusion kernel (jump to step t)

With the cumulative product $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$ (the lecture writes this as β_t),

$$q(\mathbf{x}_t \,|\, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t\,;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\, (1-\bar{\alpha}_t)\mathbf{I}).$$

So $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ for a single $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

This one-shot kernel is what makes training cheap: to get a training example at step t we just sample noise once, no chain to unroll.

8.3 The reverse process (the learned decoder)

We want to undo a step, i.e. $q(\mathbf{x}_{t-1} \,|\, \mathbf{x}_t)$. By Bayes' rule that needs the marginal $q(\mathbf{x}_t)$ and is **intractable**. But if we additionally condition on the clean image $\mathbf{x}_0$, the reverse-posterior becomes a tractable Gaussian,

$$q(\mathbf{x}_{t-1} \,|\, \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}\,;\, \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_t),\, \sigma_t^2 \mathbf{I}),$$

with mean and variance that work out in closed form from the schedule. Of course at generation time we do *not* have $\mathbf{x}_0$, so we train a network — a **U-Net** $\hat{\boldsymbol{\mu}}$, which conveniently maps an image to an image of the same size — to approximate the reverse step:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \,|\, \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}\,;\, \hat{\boldsymbol{\mu}}(\mathbf{x}_t, \boldsymbol{\theta}, t),\, (1-\alpha_t)\mathbf{I}).$$

8.4 Training: the ELBO, then a beautifully simple objective

The marginal likelihood is intractable, so — exactly like the VAE — we maximise an ELBO. Because the chain is deep, it unrolls into three kinds of term:

Diffusion ELBO

$$\log p(\mathbf{x}) \geq \underbrace{\mathbb{E}\big[\log p_{\boldsymbol{\theta}}(\mathbf{x}_0 \,|\, \mathbf{x}_1)\big]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T \,|\, \mathbf{x}_0) \,\|\, p(\mathbf{x}_T))}_{\text{prior matching}} - \sum_{t=2}^{T} \underbrace{\mathbb{E}\big[D_{\mathrm{KL}}(q(\mathbf{x}_{t-1} \,|\, \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \,|\, \mathbf{x}_t))\big]}_{\text{denoising matching}}.$$

The workhorse is the **denoising-matching** sum: at every step, make the learned reverse step match the tractable (conditioned-on-$\mathbf{x}_0$) posterior. Both are Gaussians, so each KL has a closed form and collapses to a simple squared distance between their means,

$$D_{\mathrm{KL}}(q \,\|\, p_{\boldsymbol{\theta}}) = \frac{1}{2(1-\alpha_t)} \, \|\hat{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_t)\|^2 + \text{const.}$$

Here comes the trick that made diffusion practical (DDPM, Ho et al., 2020). Rather than predicting the mean, **reparameterize the network to predict the noise** $\hat{\boldsymbol{\epsilon}}$ that was added. Substituting, the objective simplifies all the way down to a plain noise-prediction regression:

DDPM training objective

$$L(\boldsymbol{\theta}) = \frac{1}{T} \sum_{t=1}^{T} \left\| \boldsymbol{\epsilon}_t - \hat{\boldsymbol{\epsilon}}\big(\sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}_t,\ t;\ \boldsymbol{\theta}\big) \right\|^2 .$$

In words: noise an image to a random level t, and train the network to guess the noise you added.

This gives a remarkably clean training loop and a matching sampler:

Training and sampling (DDPM)

Training — repeat until converged:

1. Sample a data point $\mathbf{x}_0$ and a step $t \sim \mathrm{Uniform}\{1, \dots, T\}$.
2. Sample noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and form the noisy input $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}$.
3. Gradient step on $\|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}(\mathbf{x}_t, t; \boldsymbol{\theta})\|^2$.

Sampling — start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then for $t = T, \dots, 1$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \Big(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\, \hat{\boldsymbol{\epsilon}}(\mathbf{x}_t, t; \boldsymbol{\theta}) \Big) + \sigma_t \mathbf{z},$$

with fresh $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$ and $\mathbf{z} = \mathbf{0}$ at the last step. Each iteration nudges the image one notch toward clean.

8.5 Interpretation and what comes next

DPMs can be read as learning the **gradient of the log-density** (the "score") in image space. That connects them directly to two ideas from the previous chapter: the sampler is essentially **Stochastic Gradient Langevin Dynamics** — climb the density gradient, add noise to stay unbiased — and training is a form of **score matching**. The noise-prediction network is, up to scaling, learning where the density increases, and sampling follows that signal back to the data manifold.

Intuition & Mental Model

The relations make the whole course click into place. A DPM is a deep hierarchical VAE (same ELBO structure); its sampler is the EBM's Langevin dynamics; and in the continuous-time limit it becomes a differential equation, which links it to flows. The pieces you met separately — variational bounds, score/energy gradients, invertible transport — are three views of the same elephant.

Taking the number of steps $T \to \infty$ yields a **continuous-time** model described by a stochastic differential equation (the variance-preserving SDE of Song et al., 2020). A closely related, increasingly popular variant is **flow matching**: instead of a noising chain, define a straight-line interpolation $\mathbf{x}(t) = (1-t)\,\mathbf{x} + t\,\boldsymbol{\epsilon}$ between data and noise, and train a network $\boldsymbol{v}_{\boldsymbol{\theta}}$ to predict the **velocity field** $\boldsymbol{v}^* = \boldsymbol{\epsilon} - \mathbf{x}$ that carries one into the other. Generation is then just numerically integrating that ODE backwards (Euler steps) from noise to data — often in far fewer steps than a classic diffusion sampler, which is why flow matching now underlies several state-of-the-art generators.

9

Bayesian Deep Learning and Uncertainty

Every model we have built so far hands back a single answer. Train a classifier, feed it an image, and it tells you "cat, 92%". But that number is a *calibrated-looking lie* more often than we would like: a standard network will confidently scream "cat" at a picture of pure noise, or at a road sign it has never seen. The question this chapter asks is: can the model tell us when it *does not know*? That is the whole point of Bayesian deep learning — turning a network from something that always answers into something that can also say "I'm not sure".

9.1 Why we want uncertainty at all

The motivation is trust. If a model is going to drive a car, read an X-ray, or flag a molecule as toxic, a wrong answer delivered with full confidence is far more dangerous than a wrong answer that admits doubt. A system that knows the edge of its own competence can defer to a human, ask for more data, or abstain. So we want predictions that come with an honest *confidence*, and we want that confidence to actually rise on familiar inputs and fall on weird ones.

It helps to separate the two reasons a model might be uncertain, because they behave completely differently.

Aleatoric vs. epistemic uncertainty

Aleatoric uncertainty is noise *in the data itself* — genuine ambiguity that no amount of extra data can remove. A blurry photo that is honestly half-cat-half-dog should get a prediction near 0.5, and that 0.5 is correct.

Epistemic uncertainty is uncertainty *in the model* — it comes from not having seen enough data, and it *shrinks as you collect more*. This is the uncertainty that should spike on inputs unlike anything in training.

Intuition & Mental Model

The two can produce the same output probability for very different reasons. A clean image of an animal that is truly ambiguous gives $p \approx 0.5$ from *aleatoric* noise. A bizarre out-of-distribution input might also average out to $p \approx 0.5$ — but here it is *epistemic*: the model has no idea, and different plausible models would disagree wildly. A single softmax number cannot tell these apart. Separating them is exactly what the Bayesian treatment buys us.

9.2 From a point estimate to a posterior over weights

Ordinary training picks *one* weight vector. Maximum likelihood maximises $p(\mathcal{D} \mid \mathbf{w})$; adding a prior and maximising $p(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w})$ gives the **MAP** estimate (this is just weight-decay-style regularisation in disguise). Either way you collapse everything you know about the weights down to a single point and throw away the rest.

The Bayesian move is to refuse that collapse and keep the whole *posterior distribution* over weights:

$$p(\mathbf{w} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w})}{p(\mathcal{D})}, \qquad p(\mathcal{D}) = \int p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w})\, \mathrm{d}\mathbf{w}.$$

The numerator is easy; the trouble is the denominator. That integral — the **evidence** — runs over millions of weights and is hopelessly intractable for any real network. This single intractable integral is the reason the entire rest of the chapter exists: every practical method is a different way of dodging it.

9.3 Bayesian model averaging

Once we have a posterior, prediction is not "run the one best network". It is "ask *every* plausible network, weighted by how much the posterior believes in it, and average their votes".

Posterior predictive (Bayesian model averaging)

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int p(y^* \mid \mathbf{x}^*, \mathbf{w})\; p(\mathbf{w} \mid \mathcal{D})\, \mathrm{d}\mathbf{w}.$$

The prediction for a new input is the average of all models' predictions, each weighted by its posterior plausibility. A point estimate is the degenerate case where the posterior is a single spike.

This averaging is also where the clean split between the two uncertainties comes from. If you measure total predictive uncertainty by the entropy of the averaged prediction, it decomposes exactly:

$$\underbrace{\mathrm{H}\big[\,\mathbb{E}_{p(\mathbf{w} \mid \mathcal{D})}\, p(y \mid \mathbf{x}, \mathbf{w})\big]}_{\text{total}} = \underbrace{\mathbb{E}_{p(\mathbf{w} \mid \mathcal{D})}\, \mathrm{H}\big[p(y \mid \mathbf{x}, \mathbf{w})\big]}_{\text{aleatoric (expected entropy)}} + \underbrace{\mathrm{I}(y; \mathbf{w})}_{\text{epistemic (mutual information)}}.$$

The first term is the noise each model sees on average; the second is how much the models *disagree with each other*. Disagreement is the signature of epistemic uncertainty — and it is exactly what blows up on inputs far from the training data, where different plausible weight settings give different answers.

Of course we cannot do the integral, so in practice we draw S weight samples $\mathbf{w}^{(s)}$ from (an approximation of) the posterior and Monte-Carlo average:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} p(y^* \mid \mathbf{x}^*, \mathbf{w}^{(s)}).$$

The practical methods below are really just four different answers to the same question: *where do those weight samples come from?*

9.4 Practical methods

9.4.1 Deep Ensembles

The simplest idea works embarrassingly well: train the same network several times from different random initialisations (and shuffles), then average their predictions (Lakshminarayanan et al., 2017). The different runs land in different parts of the loss surface, so on familiar inputs they agree and on strange inputs they diverge — which is precisely the epistemic-disagreement signal we wanted. It is not Bayesian by construction, but it behaves like a coarse posterior sample and is still one of the strongest, most reliable uncertainty baselines around. The only real cost is training and storing K networks.

9.4.2 Bayes by Backprop (weight uncertainty)

Instead of a fixed weight, give *every weight its own little Gaussian*, and learn the means and variances (Blundell et al., 2015). We posit a variational approximation $q(\mathbf{w} \mid \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and fit it to the true posterior by maximising the ELBO — equivalently, minimising

$$\mathcal{L}(\boldsymbol{\theta}) = D_{\mathrm{KL}}(q(\mathbf{w} \mid \boldsymbol{\theta}) \,\|\, p(\mathbf{w})) \;-\; \mathbb{E}_{q(\mathbf{w} \mid \boldsymbol{\theta})}\big[\log p(\mathcal{D} \mid \mathbf{w})\big].$$

The KL term pulls the weight distributions toward the prior; the data term keeps them explaining the training set. To backprop through the sampling step we reuse the reparameterisation trick from the VAE chapter: $\mathbf{w} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, so gradients flow into $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$. The price is that the network now has *twice* the parameters (a mean and a variance per weight).

9.4.3 MC Dropout

Here is the cheap trick that made Bayesian deep learning practical for a lot of people: keep **dropout switched on at test time** and run the network several times (Gal & Ghahramani, 2016). Each forward pass drops a different random set of units, so each pass is effectively a different sub-network — a sample. Average the passes for your prediction and look at their spread for your uncertainty. The beautiful part is that this requires *no change to the architecture or training* if you already use dropout; Gal showed it can be interpreted as approximate variational inference (and connects to Gaussian processes). It is the default "free" uncertainty estimate, though its quality depends a lot on the dropout rate.

9.4.4 Laplace / Hessian-based approximation

This one starts from the MAP solution you already trained and fits a Gaussian *around* it. Take a second-order Taylor expansion of the log-posterior at $\mathbf{w}_{\mathrm{MAP}}$: the linear term vanishes (we are at an optimum) and the quadratic term is the Hessian, which gives a Gaussian

$$q(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w}_{\mathrm{MAP}},\, \mathbf{A}^{-1}), \qquad \mathbf{A} = \tfrac{1}{\sigma_w^2}\mathbf{I} + \tfrac{1}{\sigma_\epsilon^2}\mathbf{H},$$

where $\mathbf{H}$ is the Hessian of the data-fit term and the $\mathbf{I}$ piece comes from the prior. The covariance $\mathbf{A}^{-1}$ tells you how flat the loss is in each direction: *flat directions are uncertain directions.* The honest catch is that the full Hessian is a square matrix in the number of weights, so it is only feasible with approximations (diagonal, Kronecker-factored, last-layer only).

Exam trap

Don't mix up *where the uncertainty lives.* Bayes-by-Backprop and the Laplace approximation put a distribution **over the weights**. MC Dropout and Deep Ensembles never write down such a distribution — they get their samples *implicitly*, from random masks or from

re-training. And remember which uncertainty does what: more data shrinks **epistemic** uncertainty but leaves **aleatoric** noise untouched.

9.5 Two connections worth knowing

Infinite-width networks are Gaussian processes. Neal (1994) showed that a single-hidden-layer network with random weights, as the width goes to infinity, *is* a Gaussian process — a fully Bayesian, non-parametric model with closed-form uncertainty. This is the same infinite-width limit that reappears in the theory chapter as the Neural Tangent Kernel, and it is a recurring reason GPs keep showing up next to deep nets.

QUAM (work at JKU). A recent line from the institute (Schweighofer, Hochreiter et al., 2023) attacks the integral from a different angle: instead of sampling weights from the posterior at random, it actively *searches* for "adversarial" models — weight settings that the posterior still finds plausible but that *disagree* strongly with the current prediction. Finding such a model is strong evidence of epistemic uncertainty, and this targeted search can expose under-confidence that ordinary sampling misses.

10

Graph Neural Networks

10.1 Two streams in modern ML

There are, roughly, two philosophies in deep learning today. One says: *use fewer assumptions, more data, and let the network learn any symmetry it needs from examples* — this is the world of large transformers, ConvNeXt, CLIP. The other says: *bake the symmetry directly into the architecture*, so the model gets it for free and needs far less data. That second philosophy is **geometric deep learning**, and graph neural networks are its flagship.

Why bother building symmetries in? Because, as the slides put it bluntly, "no generalization without inductive bias". A plain MLP fed a flattened image *has* to learn from scratch that a digit is still the same digit when shifted a few pixels — it must discover translation invariance from data. A CNN never has to: the convolution is shift-equivariant by construction. Geometric deep learning is the program of doing this systematically, for whatever symmetry your data actually has.

10.1.1 Symmetries, groups, invariance, equivariance

A **symmetry** is a transformation that leaves some property of an object unchanged. The set of all symmetries of an object is closed under composition, contains an identity, and every element has an inverse — which is exactly the definition of a *group*. Translations of an image form a group; permutations of a set form a group; 3D rotations form the group $SO(3)$.

Two ways a function can respect a group G matter for us, and the difference is the single most important distinction in this chapter.

Invariance vs. equivariance

A function g is **invariant** to G if transforming the input does nothing to the output: $g(u(\mathbf{x})) = g(\mathbf{x})$ for every group action $u \in G$.
It is **equivariant** if transforming the input transforms the output *the same way*: $g(u(\mathbf{x})) = u(g(\mathbf{x}))$.

Pooling (max/mean/sum) is invariant — shuffle the inputs and the pooled value is unchanged. A convolutional layer is *equivariant* to shifts: move the object and its feature-map activations move with it. This gives the recurring **geometric DL blueprint**: stack equivariant layers (so spatial structure is preserved as we go deep), interleave local pooling, and only at the very end apply an invariant readout to collapse to a single prediction. CNNs are one instance — but the same template covers many architectures.

The 4G/5G: one blueprint, many architectures

Different domains just mean different symmetry groups plugged into the same recipe:

Architecture	Domain	Symmetry group
CNN	grid	translation
Spherical CNN	sphere / $SO(3)$	rotation $SO(3)$
GNN	graph	permutation Σ_n
Deep Sets	set	permutation Σ_n
Transformer	complete graph	permutation Σ_n
LSTM	1D grid	time warping

Note the punchline: a transformer is just a GNN on a fully-connected graph. Sets, graphs, and transformers all share the same symmetry — permutation.

10.2 Graphs and why their symmetry is permutation

A graph is a set of **nodes** (with feature vectors $\mathbf{x}_i$) joined by **edges** (optionally with their own features $\mathbf{e}_{ij}$). In ML notation a graph is the pair $(\mathbf{X}, \mathbf{A})$: a node-feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ and an adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ recording who connects to whom.

Graphs are extremely general — images are grid-graphs (pixels are nodes, neighbours are edges), time series are chain-graphs, sets are fully-connected graphs. The one feature that makes them special, and that dictates everything about the architecture, is that **there is no canonical ordering of the nodes**. If we relabel the nodes, we have the same graph. So any sensible model must be *permutation equivariant* at the node level (relabel inputs $\to$ labels of outputs permute the same way) and *permutation invariant* at the graph level (a prediction about the whole graph must not depend on the labelling).

Two prediction tasks follow naturally:

- **Node-level** — a label per node. E.g. in a social network, predict where each person likes to go on holiday.
- **Graph-level** — a single label per graph. E.g. in drug discovery, predict whether a whole molecule is toxic.

10.3 The message-passing framework

Almost every GNN is an instance of one simple idea: each node repeatedly *gathers information from its neighbours and updates itself*. Do this for a few rounds and information propagates across the graph.

Message-passing neural network (MPNN)

For node i with neighbours $N(i)$, one round $t \to t{+}1$ is

$$\mathbf{m}_i^{t+1} = \sum_{j \in N(i)} \underbrace{\phi(\mathbf{h}_i^t, \mathbf{h}_j^t, \mathbf{e}_{ij})}_{\text{message } \mathbf{m}_{ij}}, \qquad \mathbf{h}_i^{t+1} = \psi(\mathbf{h}_i^t, \mathbf{m}_i^{t+1}).$$

The **message function** ϕ and the **update function** ψ are both small MLPs. The initial state is the node feature, $\mathbf{h}_i^0 = \mathbf{x}_i$.

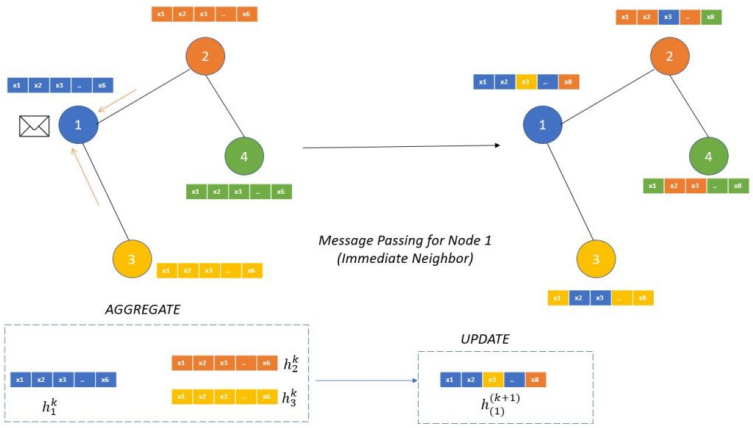

Figure 10.1: One round of message passing: a node aggregates messages $\mathbf{m}_{ij}$ from its neighbours (a permutation-invariant sum) and updates its own state. Stacking a few rounds propagates information across the graph.

Two things make this respect the graph's symmetry automatically. The neighbour sum $\sum_{j\in N(i)}$ is *permutation invariant*, so the order in which we list a node's neighbours never matters; and because every node runs the *same* ϕ and ψ, relabelling the nodes just relabels the outputs — exactly the permutation equivariance we required.

For a **graph-level** prediction we still have to crush n node vectors into one. That is the **readout**, and it must be permutation invariant, so again it is a sum/mean/max followed by a network:

$$\hat{y} = R(\{\mathbf{h}_i : i \in G\}), \qquad \text{e.g.} \quad \hat{y} = \mathbf{w}^{\mathsf{T}}\Big(\tfrac{1}{|G|}\textstyle\sum_{\nu\in G}\mathbf{h}_\nu\Big).$$

10.3.1 The Graph Laplacian view and the link to transformers

There is a second, "spectral" way to write a GNN that turns out to be the same thing. With degree matrix $\mathbf{D}$, the normalised **graph Laplacian** is $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$. The famous graph convolutional network (Kipf & Welling, 2016) folds message and update into a single matrix expression

$$\mathbf{H}^{l+1} = \sigma\big(\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}\,\mathbf{H}^{l}\mathbf{W}^{l}\big),$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ adds self-loops. Written this way the connection to transformers is obvious: it has the shape $\mathbf{H}^{l+1} = \mathbf{A}\,(\mathbf{H}^{l}\mathbf{W}^{l})$, and if you let the graph be fully connected and *learn* the entries of $\mathbf{A}$ from the data, the "adjacency" becomes an **attention matrix**. CNNs (fixed local weights), transformers (learned attention weights), and MPNNs (learned messages) are three points on one spectrum of how a node mixes information from its neighbours.

10.3.2 Building in more than permutation: EGNN

For data like molecules in 3D, each node also has *coordinates*, and we want the network equivariant to rotations and translations too. The E(n)-equivariant GNN (Satorras et al.) achieves this with a clever trick: messages depend only on the *squared distance* $\big\|\mathbf{x}_i^t - \mathbf{x}_j^t\big\|^2$ (which no rotation or translation can change), and coordinates are updated along the *difference vectors*:

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + C\sum_{j\neq i}(\mathbf{x}_i^t - \mathbf{x}_j^t)\,\phi_1(\mathbf{m}_{ij}).$$

Each layer is rotation/translation equivariant and the whole network is invariant — E(n)-equivariant by construction, no data augmentation needed.

10.4 What goes wrong with GNNs

Message passing is elegant but has four characteristic failure modes, all worth recognising:

- **Oversmoothing** — stack too many rounds and every node's representation converges to the same vector. The graph blurs into mush.
- **Oversquashing** — a node's influence on a distant node decays exponentially with their shortest-path distance, because all that long-range information must be squeezed through narrow bottlenecks. This is the GNN version of the vanishing-gradient problem.
- **Under-reaching** — if a task needs information from a node k hops away but you only run fewer than k message-passing steps, that information simply never arrives.
- **Limited expressivity** — there are genuinely different graphs that a standard GNN *cannot tell apart*, no matter how it is trained.

10.4.1 Expressivity: the Weisfeiler–Lehman ceiling

That last point has a precise theory. The **1-WL test** (also called colour refinement) is a classic algorithm for telling graphs apart: hash each node's features into a colour, then repeatedly re-hash each node's colour together with the multiset of its neighbours' colours, until the colouring stabilises. Two graphs with different final colour histograms are definitely not isomorphic.

The expressivity ceiling

A standard message-passing GNN is **at most as powerful as the 1-WL test**. If 1-WL cannot distinguish two graphs, neither can the GNN — so e.g. a 6-cycle and two separate triangles look identical to it. More powerful variants (k-WL) exist but cost more. This is the fundamental expressivity limit of MPNNs.

The **choice of aggregation function** sits right at the heart of this. With identical neighbour features, *mean* and *max* both throw away how *many* neighbours there are, so they fail to separate graphs that differ only in degree — whereas **sum** keeps the count and is the most expressive choice. The catch is that summing over neighbours makes activations grow with degree and tends to explode gradients. JKU's own fix (Schneckenreiter et al., 2024) is **variance-preserving aggregation** (VPA), a $1/\sqrt{N}$-scaled sum that keeps the expressivity of the sum while behaving like the mean for signal propagation — the best of both.

10.5 Applications

Because "graph" is such a general data type, GNNs show up almost everywhere: chemistry and drug design (molecules are graphs), protein biology (3D structures as point clouds, handled by SE(3)-invariant transformers), recommender systems and social networks, traffic forecasting, particle physics, and more. The institute's own recent work fits the same mould — VN-EGNN adds virtual nodes to an E(3)-equivariant GNN to find protein binding sites, Universal Physics Transformers scale neural operators for simulation, and diffusion models on graphs tackle combinatorial optimisation. The common thread is always the same: identify the symmetry of the data, then build it in.

11

Theory

This final chapter steps back from specific architectures and asks the harder questions. *Why* can neural networks represent anything at all? *Why* can we train ones that are a hundred layers deep when naively they should suffer hopeless gradient problems? And *why*, given that the loss surface is wildly non-convex, does gradient descent reliably find good solutions anyway? The answers are partial and sometimes hand-wavy, but they are illuminating.

11.1 Universal approximation

The oldest theoretical result is also the most reassuring: a neural network can represent essentially any function you want.

Universal approximation (Hornik, Funahashi, 1989)

If the activation f is non-constant, bounded, monotonically increasing and continuous, then any continuous function h on a compact set K can be approximated arbitrarily well by a *single-hidden-layer* network $g(\mathbf{x}; \mathbf{w})$:

$$\max_{\mathbf{x} \in K} \left| g(\mathbf{x}; \mathbf{w}) - h(\mathbf{x}) \right| < \epsilon,$$

for any $\epsilon > 0$, where the number of hidden units H depends on ϵ and h.

The full proofs lean on the Hahn–Banach and Riesz representation theorems, but the intuition is friendlier. Take a smooth target function and write it as a Fourier series — a sum of cosines. Now approximate each cosine by a *staircase* of step functions. Each step is exactly what a single hidden unit with a step-like activation can produce, so a wide enough layer of such units can build up the staircase, hence the cosine, hence the whole function.

What this does and doesn't say

Universal approximation says one hidden layer is *enough in principle*. It says nothing about *how wide* (the unit count can blow up), nor whether SGD will *find* that solution. Depth and good optimisation are about efficiency and trainability — which is what the rest of the chapter is really about.

11.2 Why deep networks are trainable: Jacobians near 1

Recall the vanishing/exploding gradient problem. Backpropagated errors obey the recursion

$$\boldsymbol{\delta}^{[l-1]\,\mathsf{T}} = \boldsymbol{\delta}^{[l]\,\mathsf{T}} \underbrace{\mathbf{W}^{[l]}\,\mathrm{diag}(f'(\mathbf{s}^{[l-1]}))}_{\mathbf{J}},$$

so at every layer the error vector is multiplied by a Jacobian $\mathbf{J}$. Iterate this over many layers and the error norm shrinks or grows *exponentially* unless the Jacobian's **singular values stay close to 1**. With sigmoids the derivative is at most 0.25, so gradients vanish; this is precisely why training deep nets used to be so hard.

The architectures that broke the depth barrier all share one trick: a **skip connection** that makes the Jacobian look like identity-plus-something.

$$\text{ResNet: } \mathbf{y} = \mathbf{x} + F(\mathbf{x}) \;\Rightarrow\; \mathbf{J} = \mathbf{I} + \mathbf{F},$$

and Highway networks and the LSTM cell-state recursion have the same $\mathbf{I} + \mathbf{F}$ form. Why does this help? The **Ky-Fan inequalities** bound the singular values: $s(\mathbf{I}+\mathbf{F})$ is squeezed into roughly $1 \pm s(\mathbf{F})$, so they *concentrate around 1.* Gradients neither vanish nor explode, and suddenly you can train through a hundred layers. This is the theoretical reason residual connections work.

11.2.1 Four features good deep nets share

Generalising from that, well-behaved deep networks tend to have four properties:

- **Near volume conservation** — Jacobian singular values concentrated around 1 (the point just made).
- **Angle preservation** — so the signal does not collapse onto a single dominant neuron.
- **Faster learning in the lower layers** — observed empirically when training deep CNNs.
- **Avoidance of drift** — activations should not creep ever upward layer by layer; normalisation schemes like SELU keep the mean activation near 0.

Two of these have clean characterisations. From an information-theoretic angle, the change-of-variables theorem says that mapping $Y = h(X)$ changes the entropy by exactly $\mathrm{H}(Y) = \mathrm{H}(X) + \mathbb{E}[\log|\det \mathbf{J}_h|]$, so a layer **conserves volume** (and information) precisely when $|\det \mathbf{J}| = 1$ — the same Jacobian-determinant condition that underlies normalizing flows. And a mapping $\mathbf{x} \mapsto \mathbf{Jx}$ **preserves angles** if and only if *all singular values of* $\mathbf{J}$ *are equal* (then $\mathbf{J} = \lambda\mathbf{O}$ for an orthogonal $\mathbf{O}$, which keeps all inner products). **Drift**, by contrast, is the failure mode: with the wrong activation the mean activation creeps up layer after layer (the "bias shift" problem), getting worse with depth — which is exactly what self-normalising activations are designed to prevent.

11.3 Random matrix theory

How do we reason about a freshly-initialised network, whose weights — and hence Jacobians — are random matrices? With **random matrix theory**, which describes the distribution of eigenvalues and singular values of large random matrices. Two results frame the picture:

- **Circular law** — the eigenvalues of a large $n \times n$ matrix with iid zero-mean, unit-variance entries (scaled by $1/\sqrt{n}$) fill the unit disk uniformly as $n \to \infty$.
- **Marchenko–Pastur** (quarter-circular) law — the *singular* values of a random rectangular matrix follow a known density on $[0, 2]$.

These let us predict how gradients flow through a random network before any training, and motivate initialisation schemes that put the singular values where we want them (near 1). The usual tools are the *method of moments* (relate $\mathbb{E}[\mathrm{tr}(\mathbf{X}^{2k})]$ to the eigenvalue distribution) and the *Stieltjes transform* of the empirical spectral density.

11.4 The loss surface

The loss of a deep network is non-convex with astronomically many critical points, so it is genuinely surprising that gradient descent works. Two complementary analyses explain why.

The first is statistical-physics flavoured. A ReLU network, for a fixed input, is a *polynomial* in its weight matrices (degree equal to the number of layers), and under some assumptions (weights on a sphere) it can be modelled as a **spherical spin glass**. In that model critical points correspond to energy levels, and crucially *high-energy (bad) critical points are exponentially unlikely* — most of the local minima you actually encounter are good, and sit at roughly the same low loss.

The second result makes this exact for the linear case.

Loss surface of deep linear networks (Kawaguchi, 2016)

For a deep network with $L \geq 2$ layers, $\hat{\mathbf{y}} = \mathbf{W}^{[L]} \cdots \mathbf{W}^{[1]}\mathbf{x}$, the squared-error loss surface has:

- the objective is *non-convex and non-concave*;
- *every local minimum is a global minimum*;
- every critical point that is not a global minimum is a *saddle point*;
- under a rank condition on the product of upper layers, every such saddle has a direction of negative curvature — so gradient descent can escape it.

The takeaway is encouraging: there are no *bad local minima* to get trapped in, only saddle points, and those have escape routes. (And finding the exact global minimum is not even desirable in practice — it would mean overfitting the training set.)

11.5 Outlook

Several deeper topics were named but left for further study: **flat minima and the Fokker–Planck view of SGD** (treating SGD as noisy diffusion that prefers wide, flat basins, which tend to generalise better), the **Neural Tangent Kernel** (the infinite-width limit where training becomes kernel regression — the same limit that turns networks into the Gaussian processes from the Bayesian chapter), and **neural ODEs** (treating a residual network as a continuous-time dynamical system). And beyond pure theory, the field keeps expanding into few- and zero-shot learning, meta-learning, domain adaptation, adversarial robustness, self-supervised and contrastive learning, energy-based models, and more — the map is still being drawn.